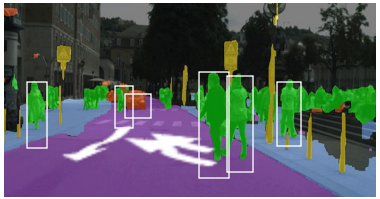
Deep Learning for Autonomous Vehicles

Lecture 3: Perception

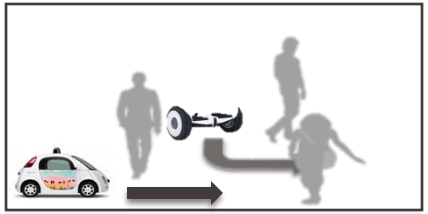
A. Alahi

Last week



Social Forecasting

Perceiving



Planning

Last week

- New functional for the score function f(x,W)
  - Changing representation x
  - Learning representation
  - Neural Network
  - Convolutional Neural Networks
  - Regularization

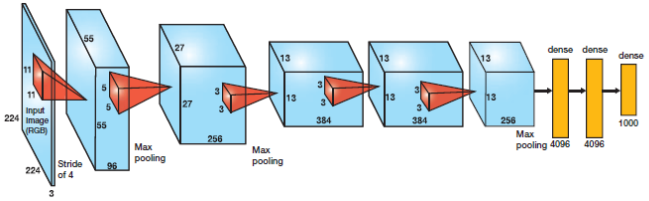
Today

- It s all about the **Gradient!**
  - Quick review of Backpropagation
  - Tip & Tricks to implement Neural Networks

Backpropagation
- Backprop -

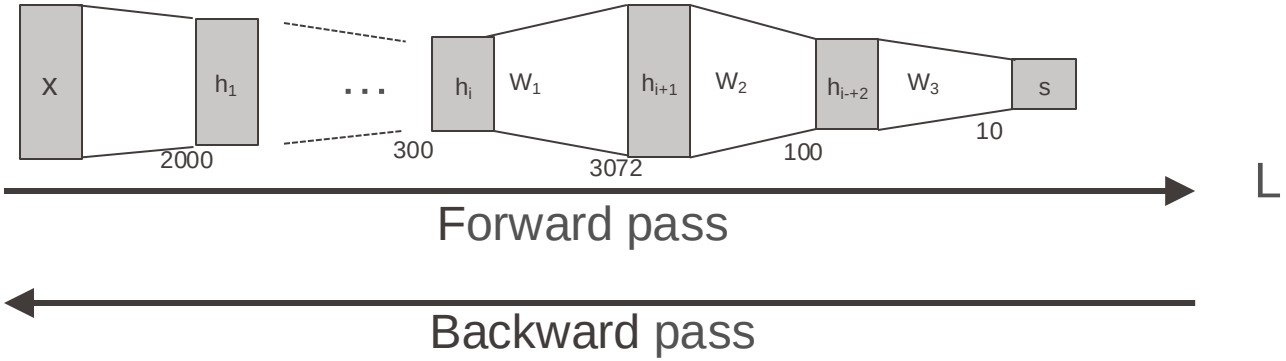
A. Alahi

Backprop



- Neural networks are in practice deep (many layers)
  - ⇒Impractical to write gradient formula by hand
- We will take advantage of **chain rule**

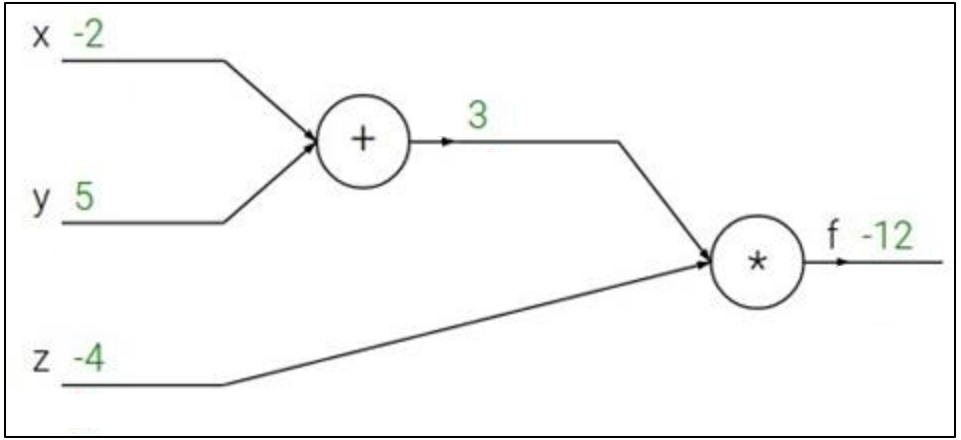
(f o g)' = (f' o g) g'



Backprop

$$f(x, y, z) = (x + y)z$$

e.g., x = -2, y = 5, z = -4



Backprop

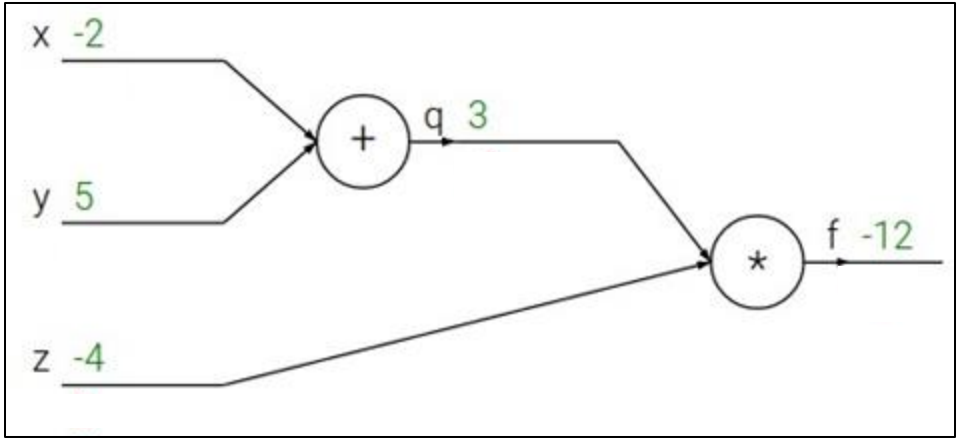
$$f(x, y, z) = (x + y)z$$

e.g., x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backprop

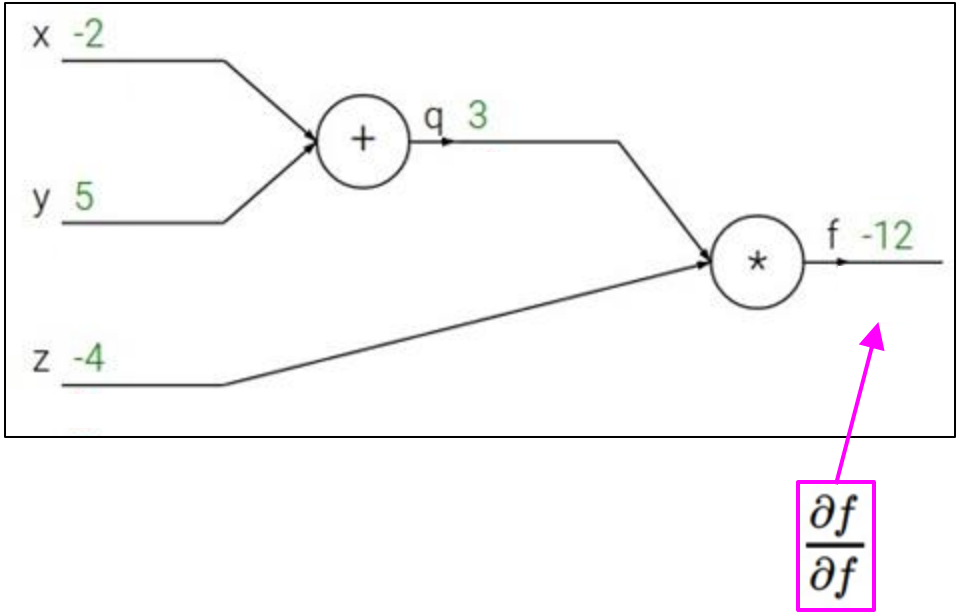
$$f(x, y, z) = (x + y)z$$

e.g., x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backprop

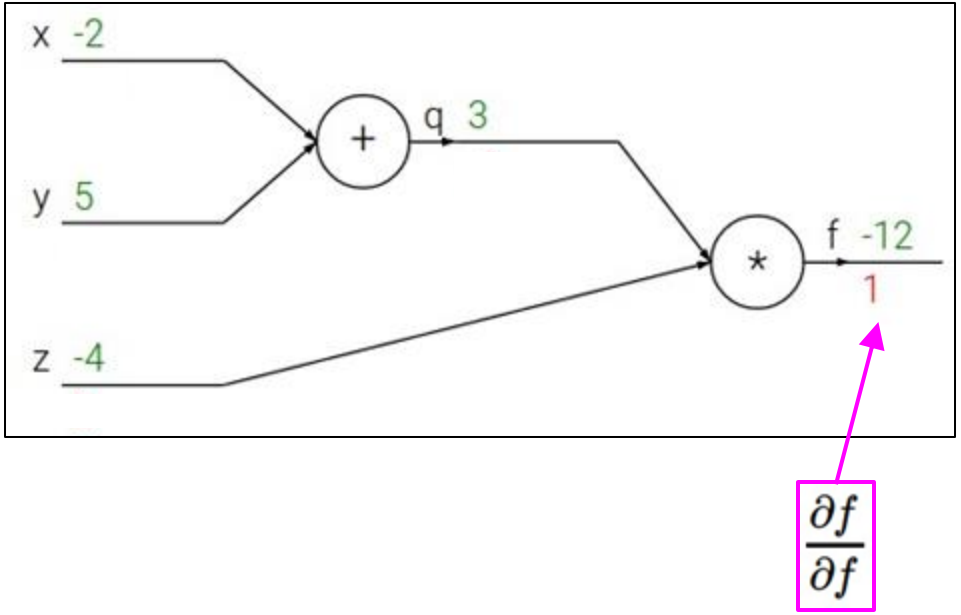
$$f(x, y, z) = (x + y)z$$

e.g., x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backprop

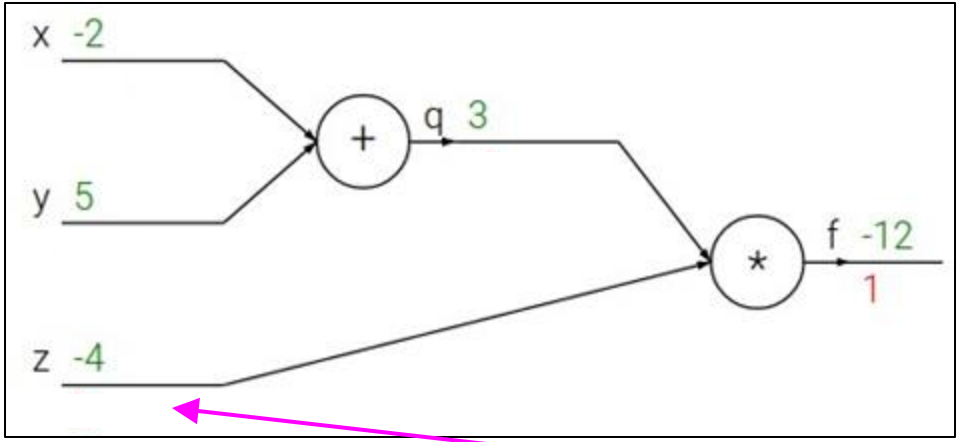
$$f(x, y, z) = (x + y)z$$

e.g., x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

Backprop

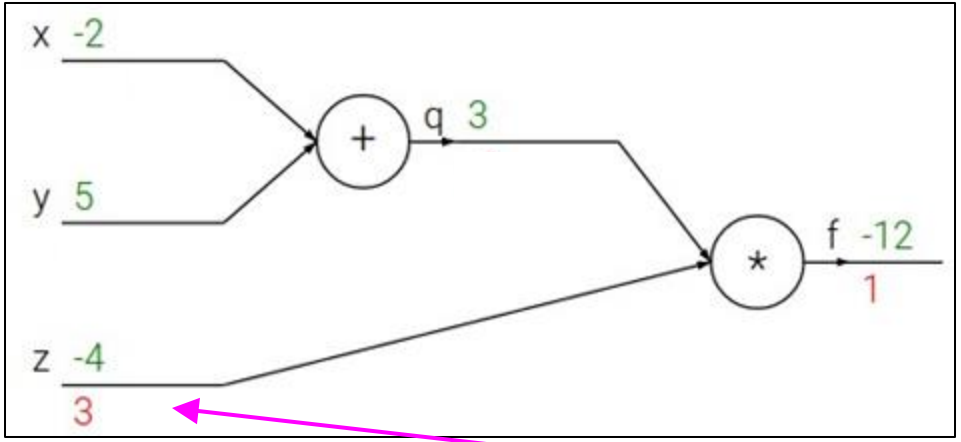
$$f(x, y, z) = (x + y)z$$

e.g., x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

Backprop

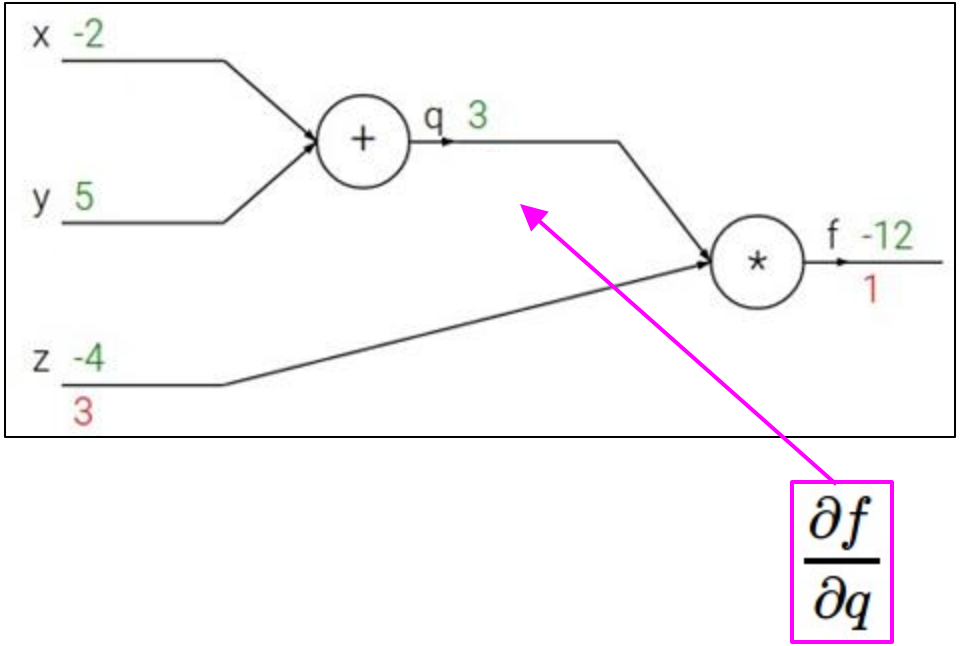
$$f(x, y, z) = (x + y)z$$

e.g., x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



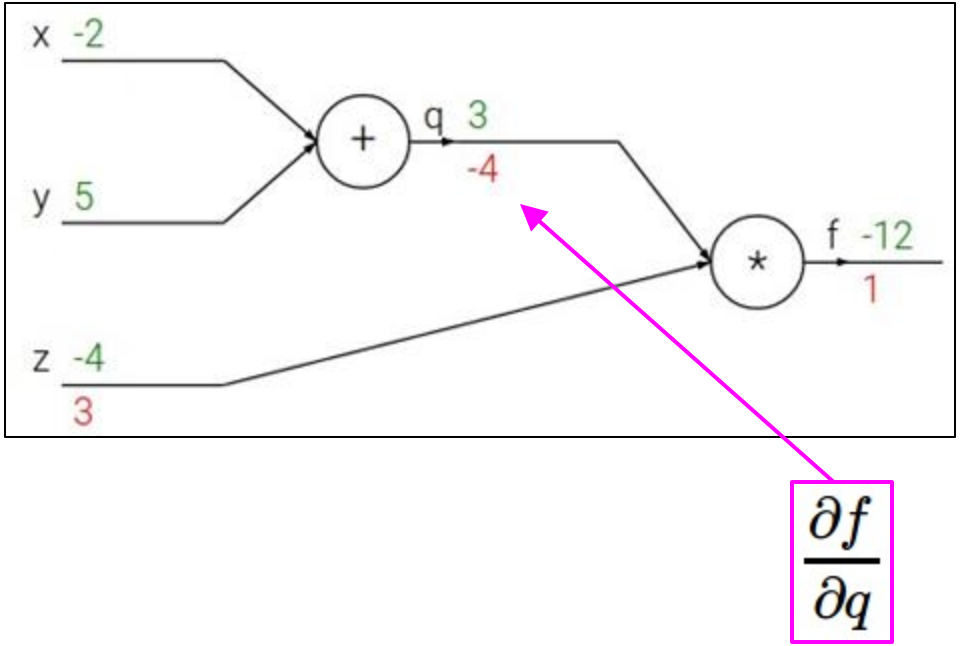
$$f(x, y, z) = (x + y)z$$

e.g., x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backprop

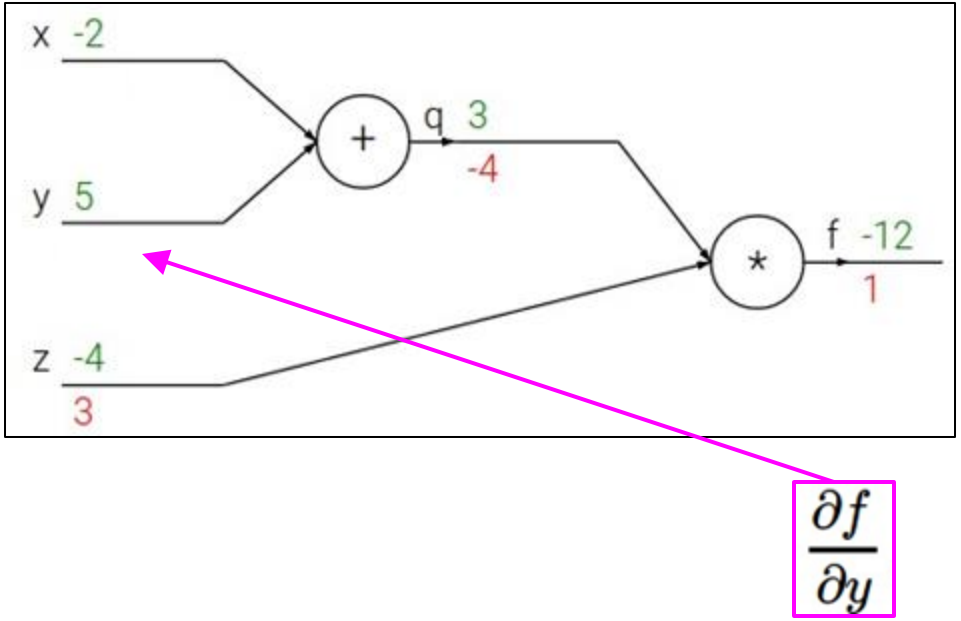
$$f(x, y, z) = (x + y)z$$

e.g., x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backprop

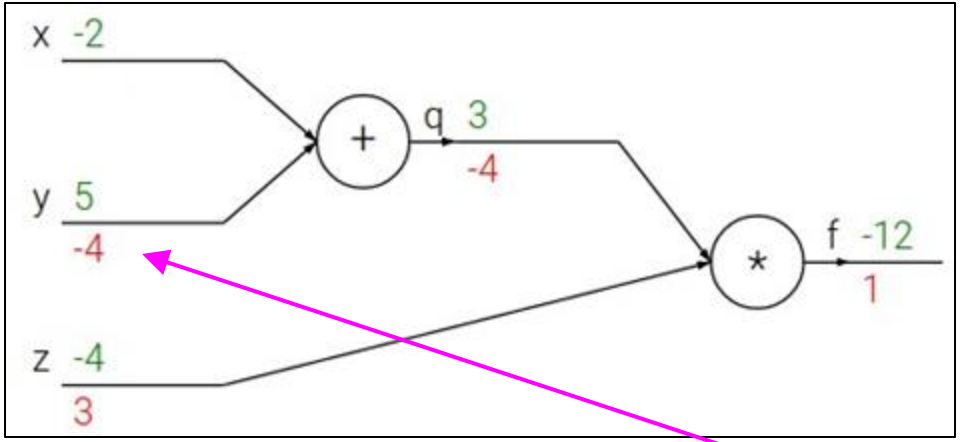
$$f(x, y, z) = (x + y)z$$

e.g., x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Backprop

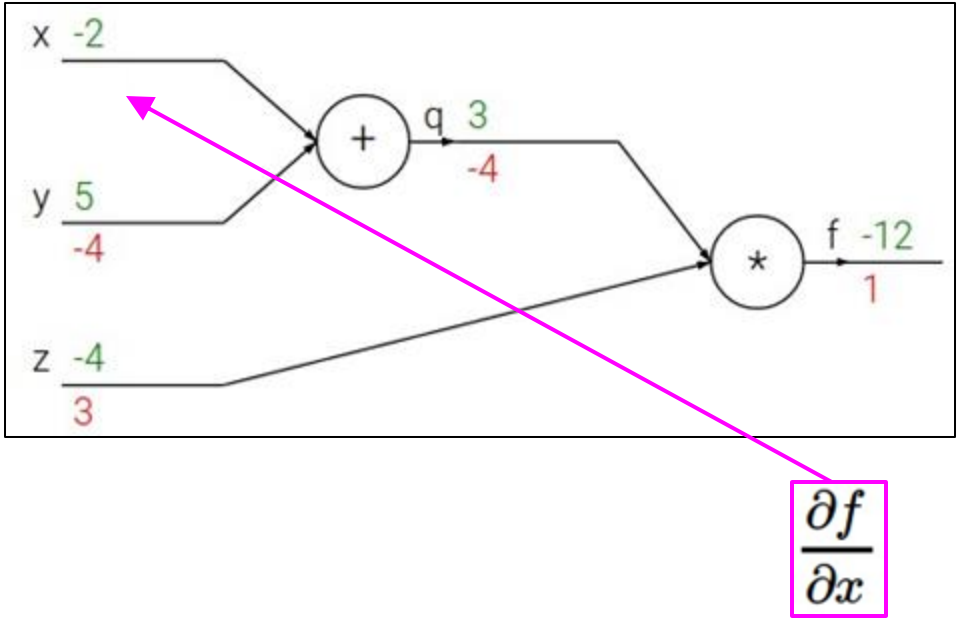
$$f(x, y, z) = (x + y)z$$

e.g., x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backprop

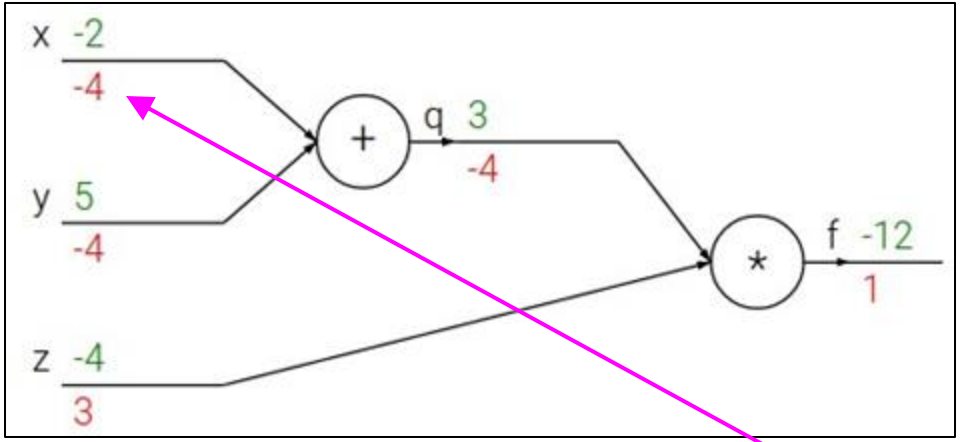
$$f(x, y, z) = (x + y)z$$

e.g., x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

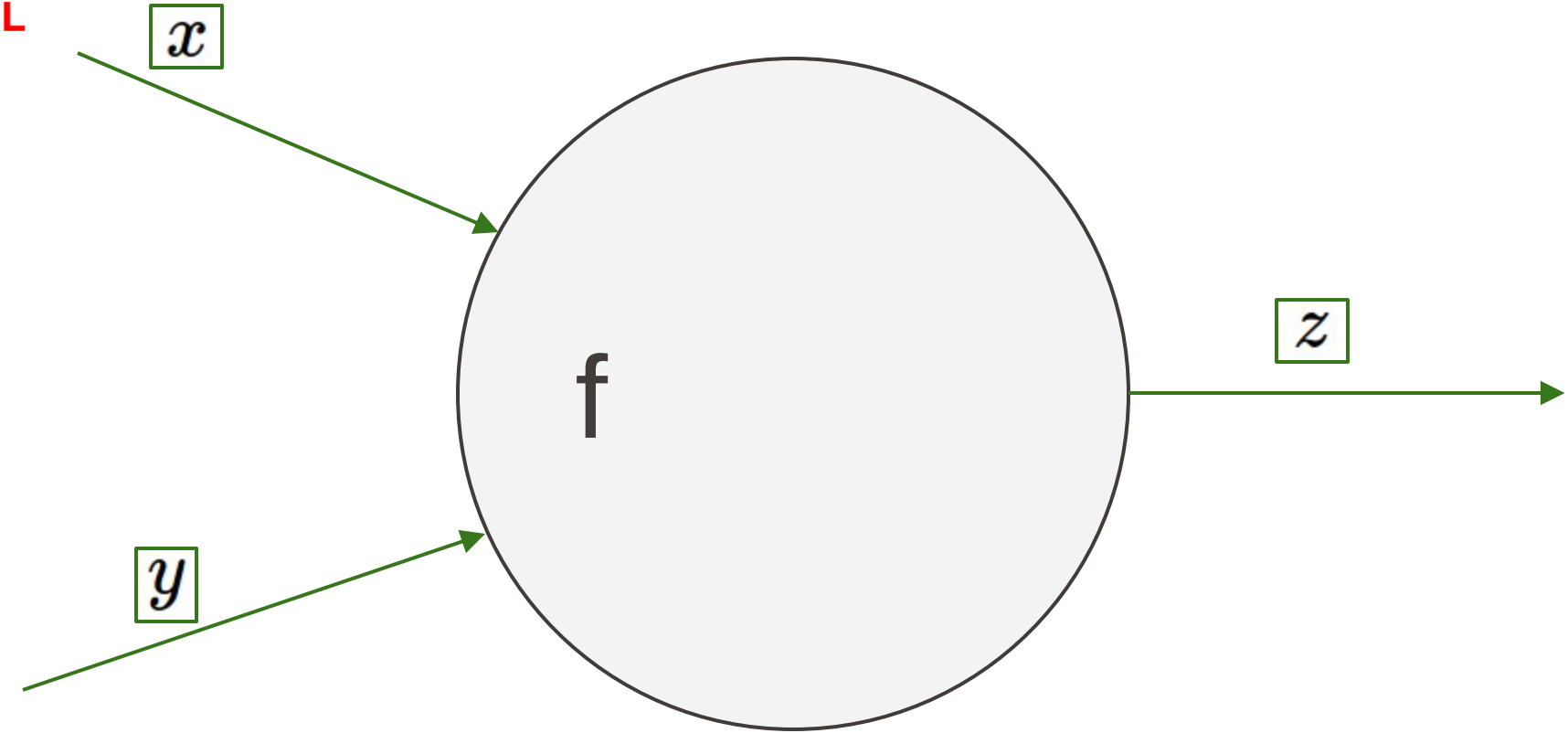
Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



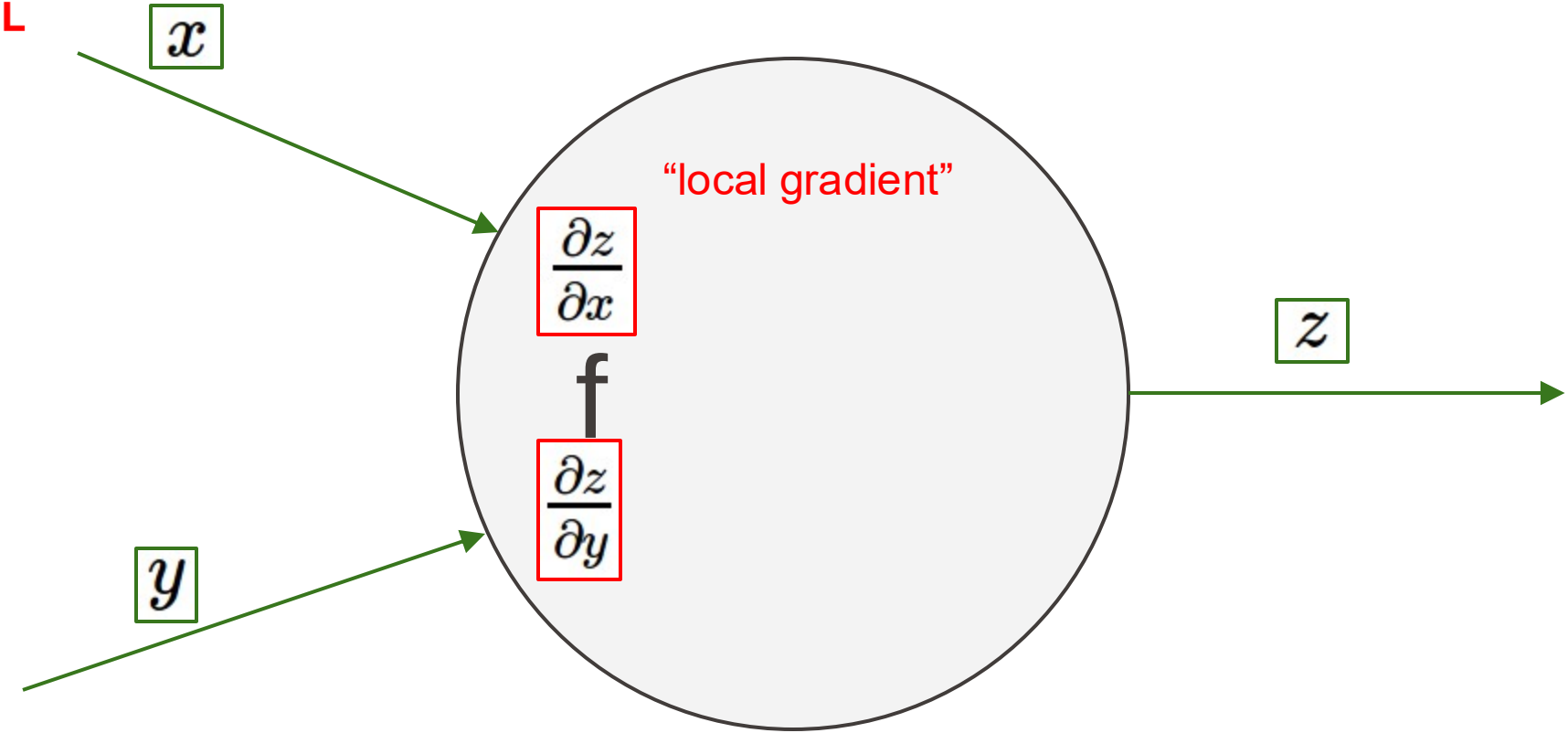
$$\frac{\partial f}{\partial x}$$

Chain rule:

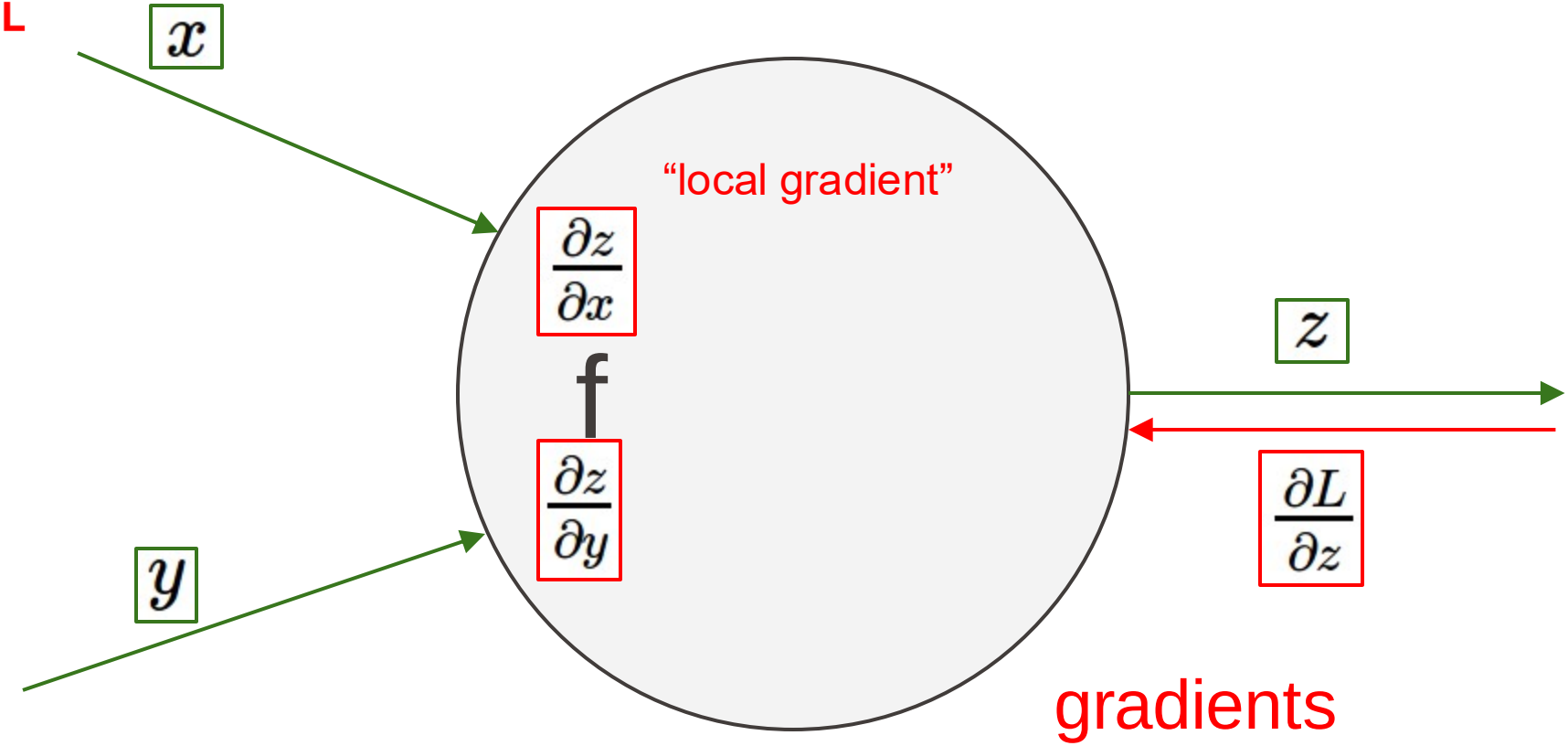
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$



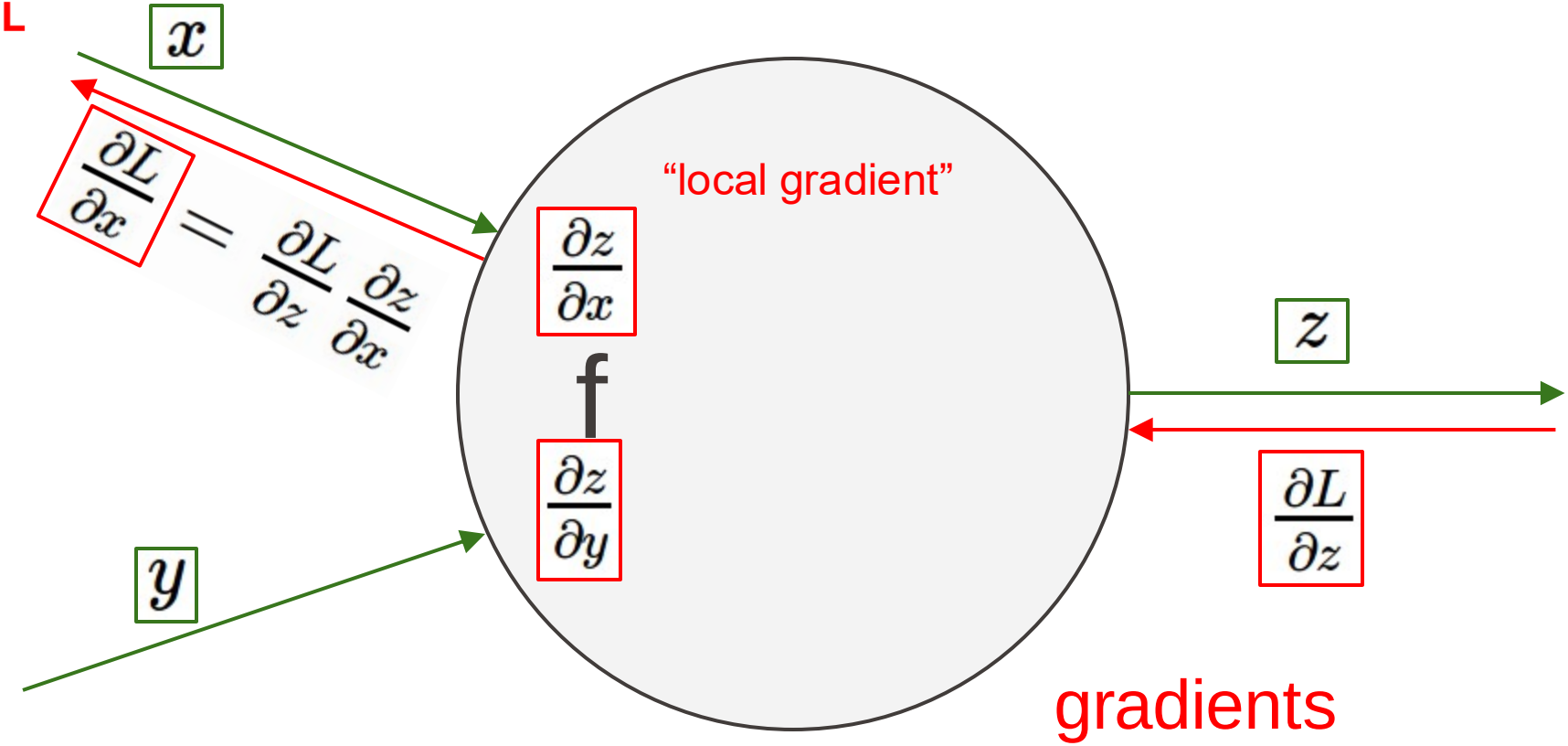
A. Alahi


x
y
$\frac{\partial z}{\partial x}$
f
$\frac{\partial z}{\partial y}$
"local gradient"
z

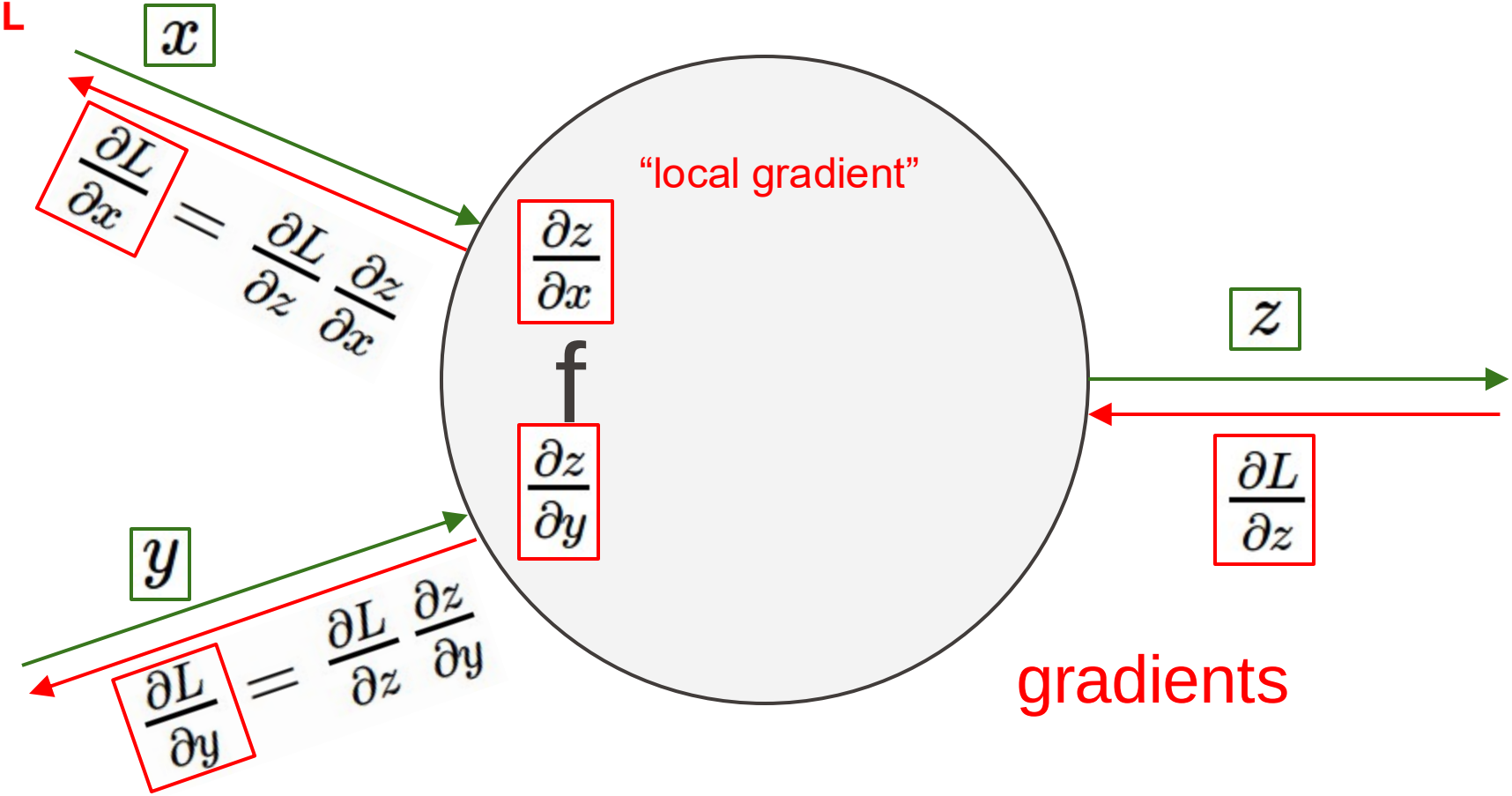
A. Alahi


x
y
$\frac{\partial z}{\partial x}$
f
$\frac{\partial z}{\partial y}$
"local gradient"
z
$\frac{\partial L}{\partial z}$
gradients

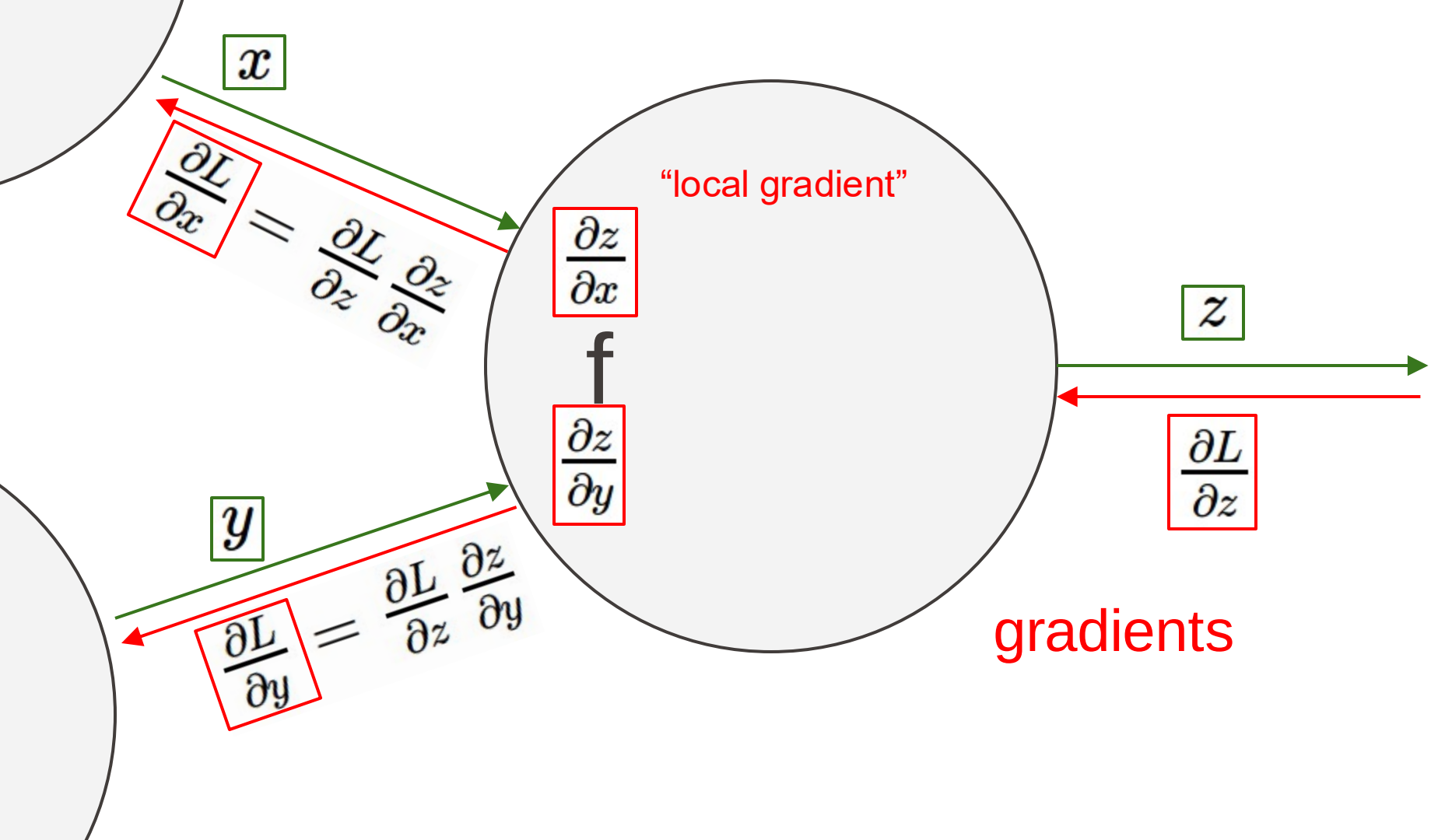
A. Alahi


x
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
"local gradient"
$\frac{\partial z}{\partial x}$
f
$\frac{\partial z}{\partial y}$
y
z
$\frac{\partial L}{\partial z}$
gradients

A. Alahi


x
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
"local gradient"
$\frac{\partial z}{\partial x}$
f
$\frac{\partial z}{\partial y}$
z
$\frac{\partial L}{\partial z}$
y
$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial y}$
gradients

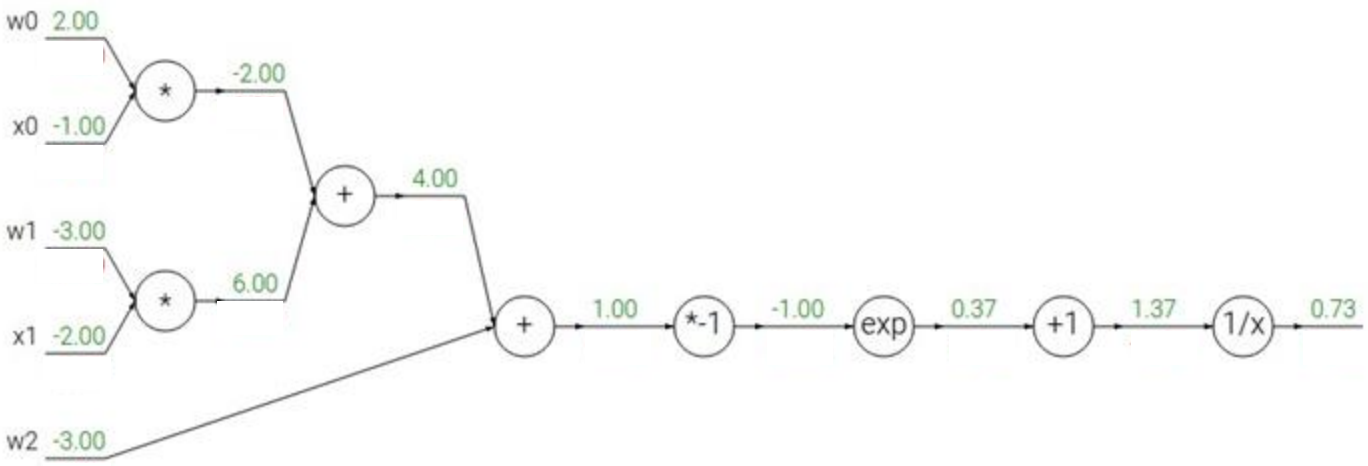
A. Alahi


x
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
"local gradient"
$\frac{\partial z}{\partial x}$
f
$\frac{\partial z}{\partial y}$
z
$\frac{\partial L}{\partial z}$
y
$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial y}$
gradients

A. Alahi

Backprop

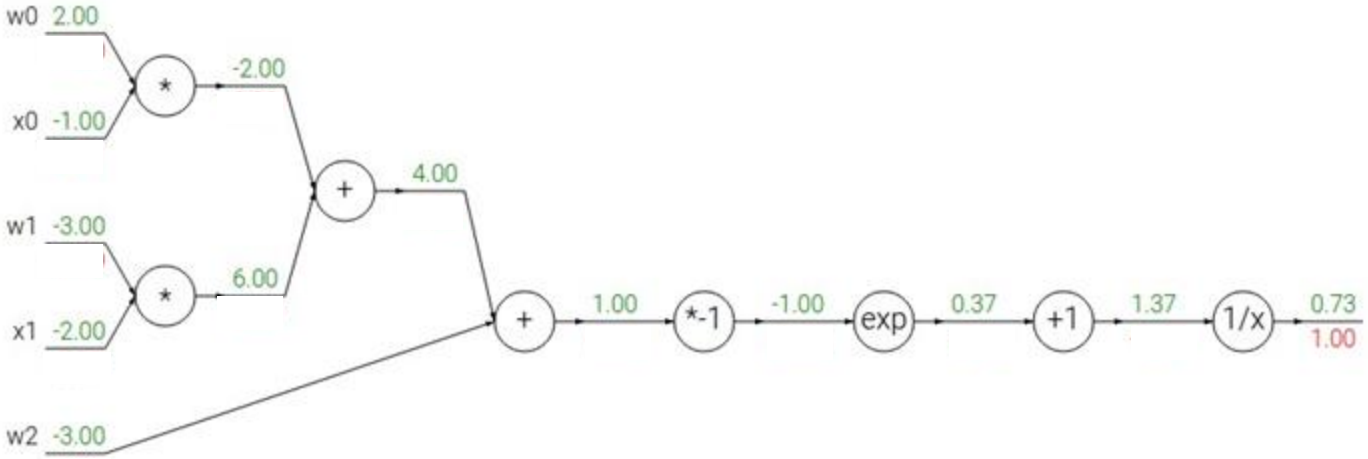
$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



A. Alahi

Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

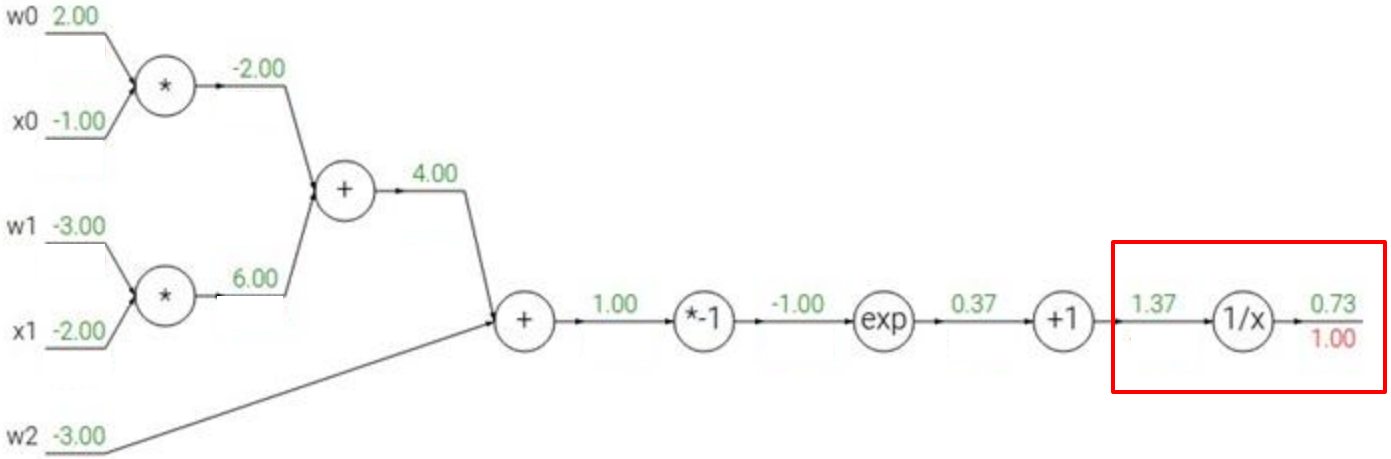
$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

A. Alahi

Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

A. Alahi



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

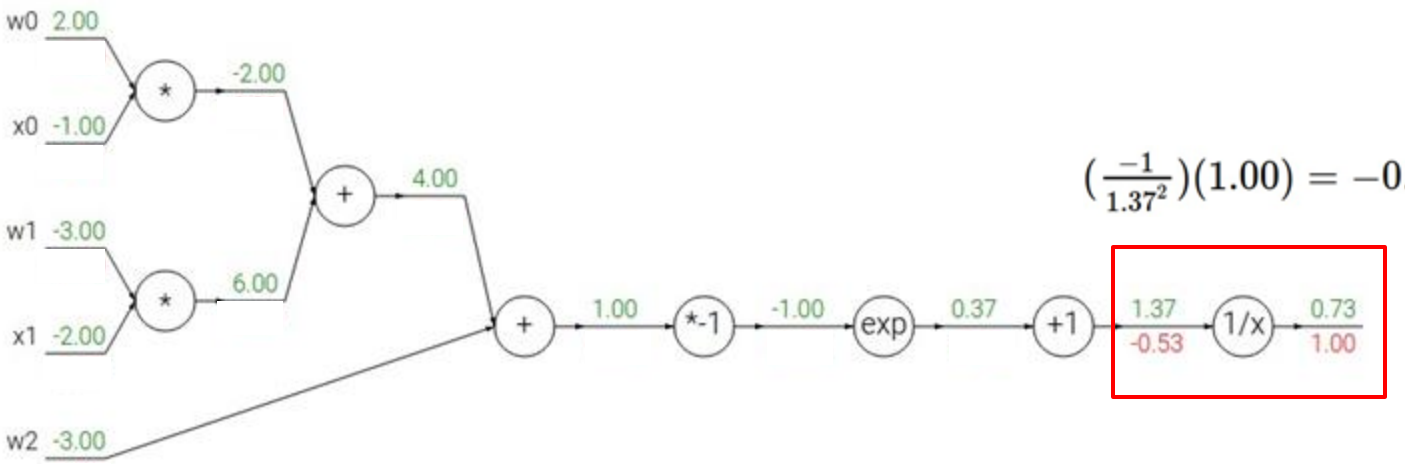
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

A. Alahi



$$\left(\frac{-1}{1.37^2}\right)(1.00) = -0.53$$

$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

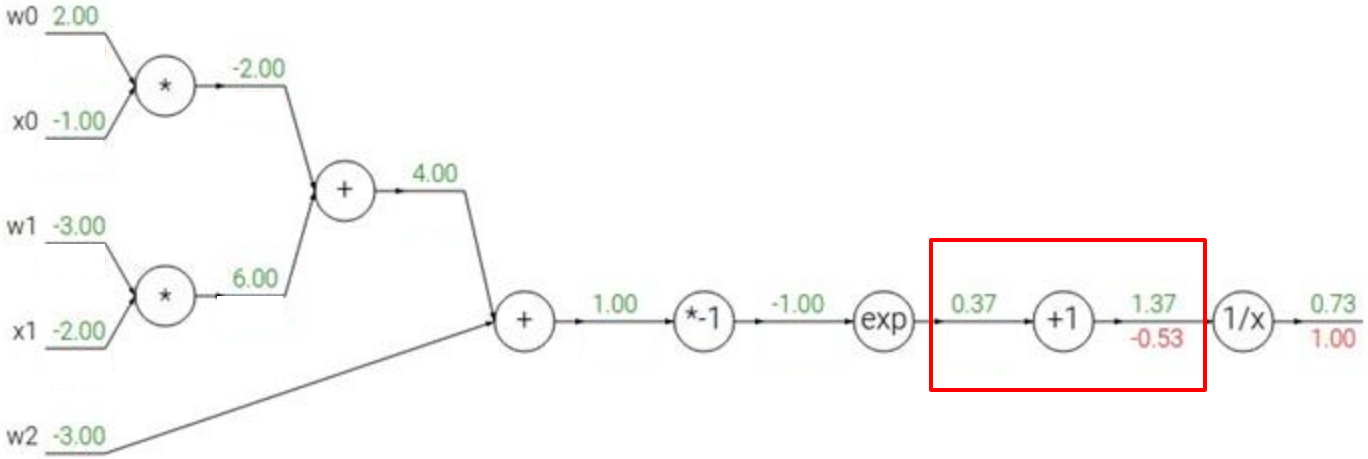
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

A. Alahi



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

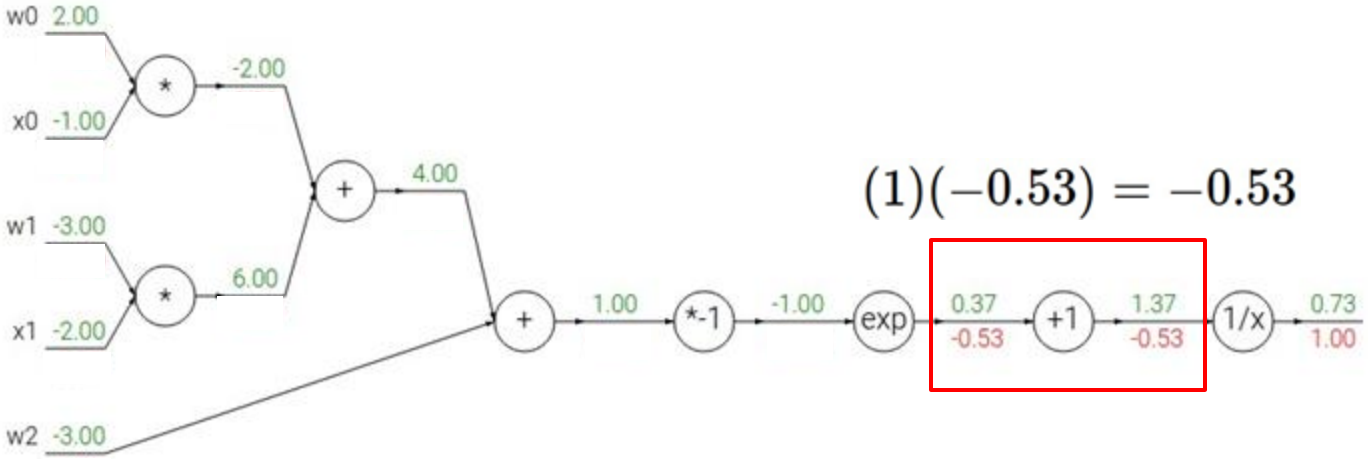
$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$(1)(-0.53) = -0.53$$

$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

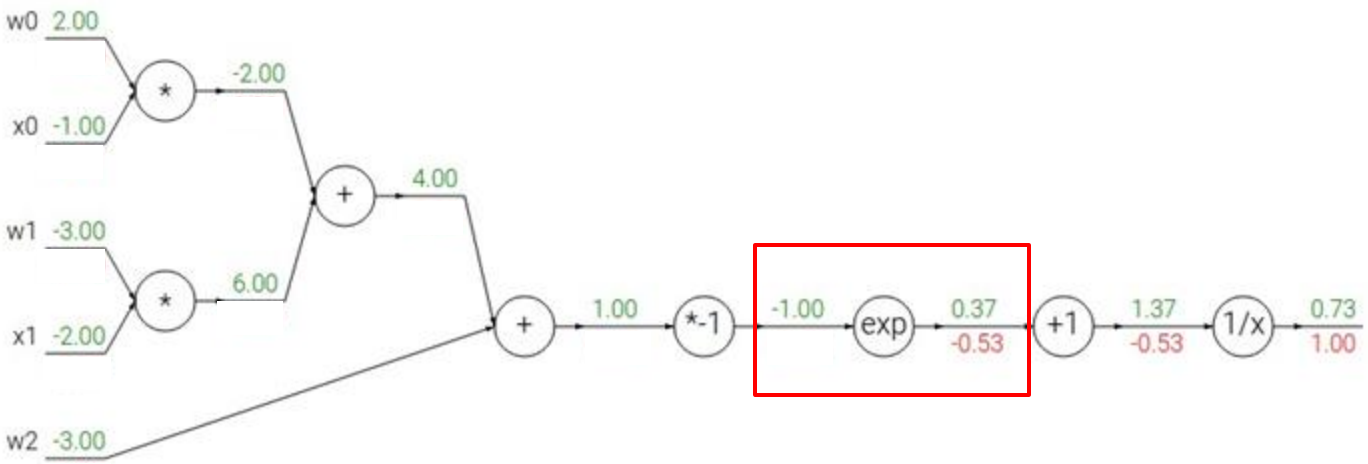
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

A. Alahi

Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

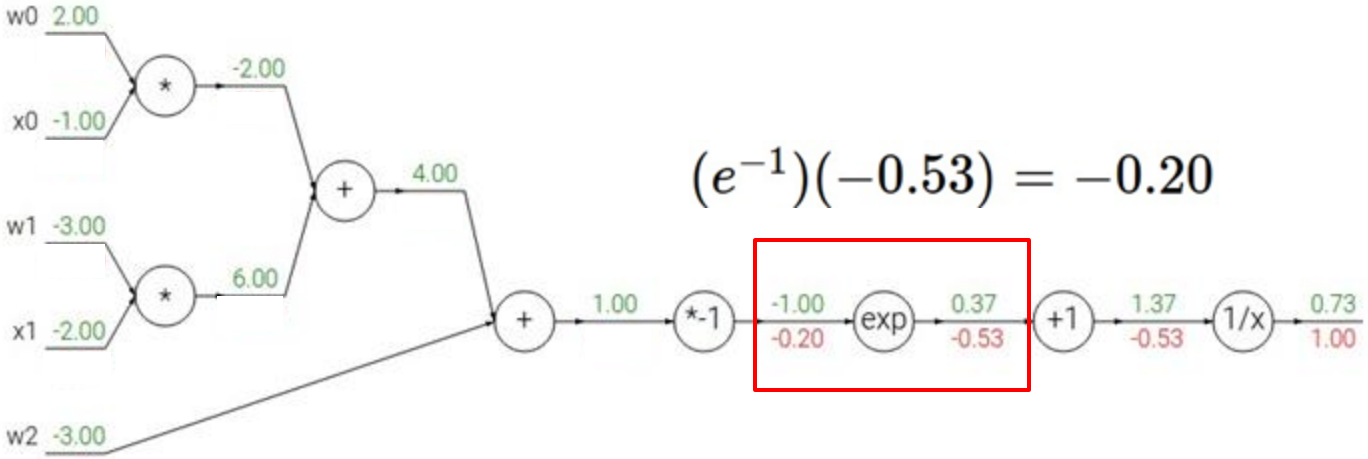
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

A. Alahi

Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$(e^{-1})(-0.53) = -0.20$$

$$\boxed{f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x}$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

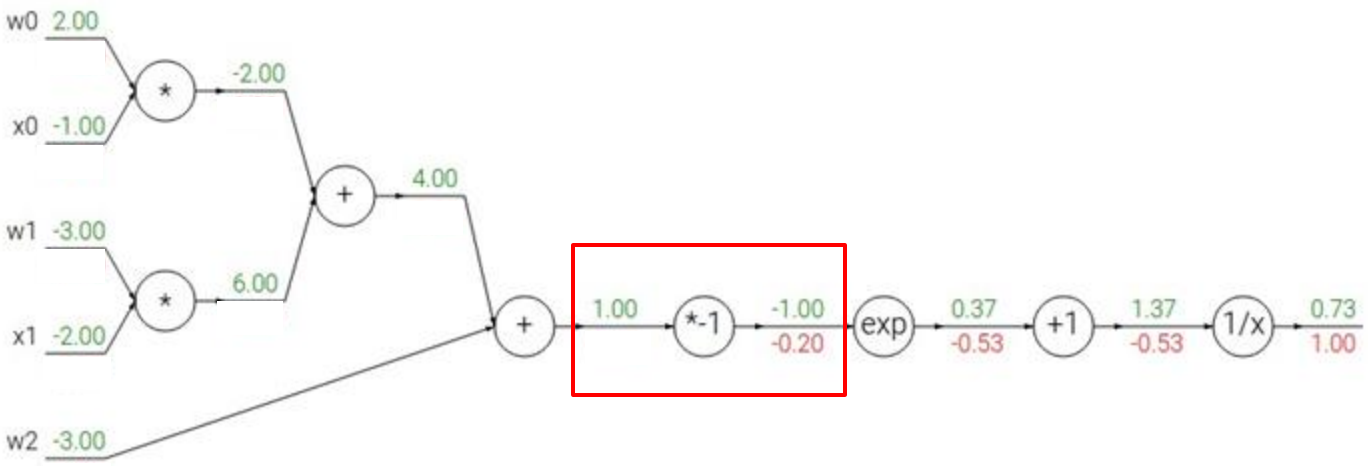
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

A. Alahi

Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

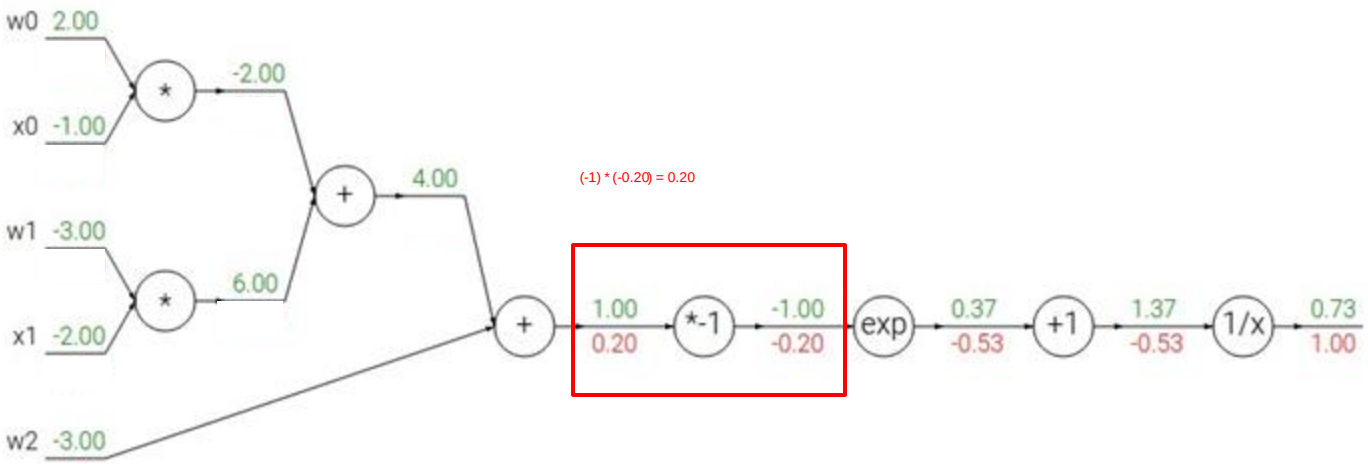
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

A. Alahi

Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



(-1) * (-0.20) = 0.20

$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

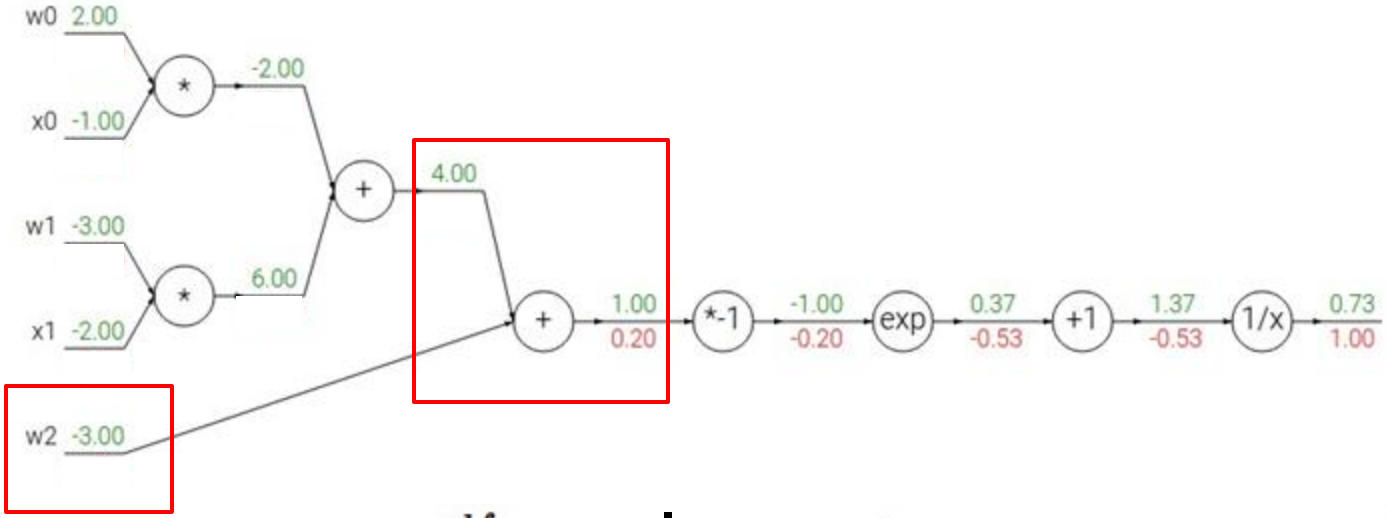
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

A. Alahi

Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

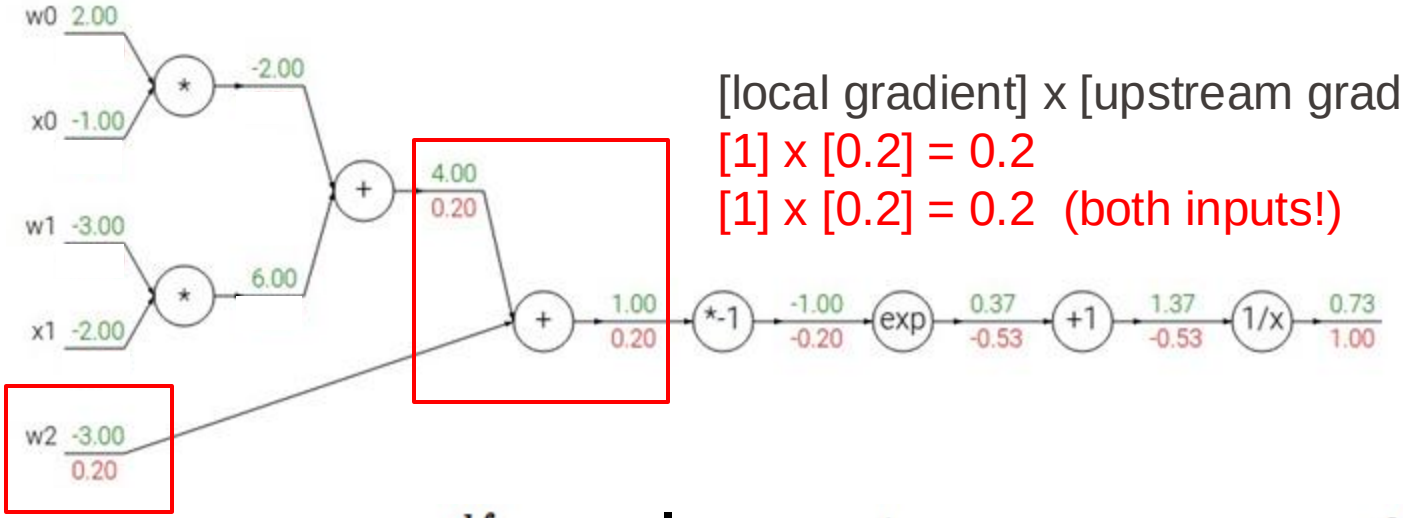
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

A. Alahi

Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



[local gradient] x [upstream gradient]

[1] x [0.2] = 0.2

[1] x [0.2] = 0.2 (both inputs!)

$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

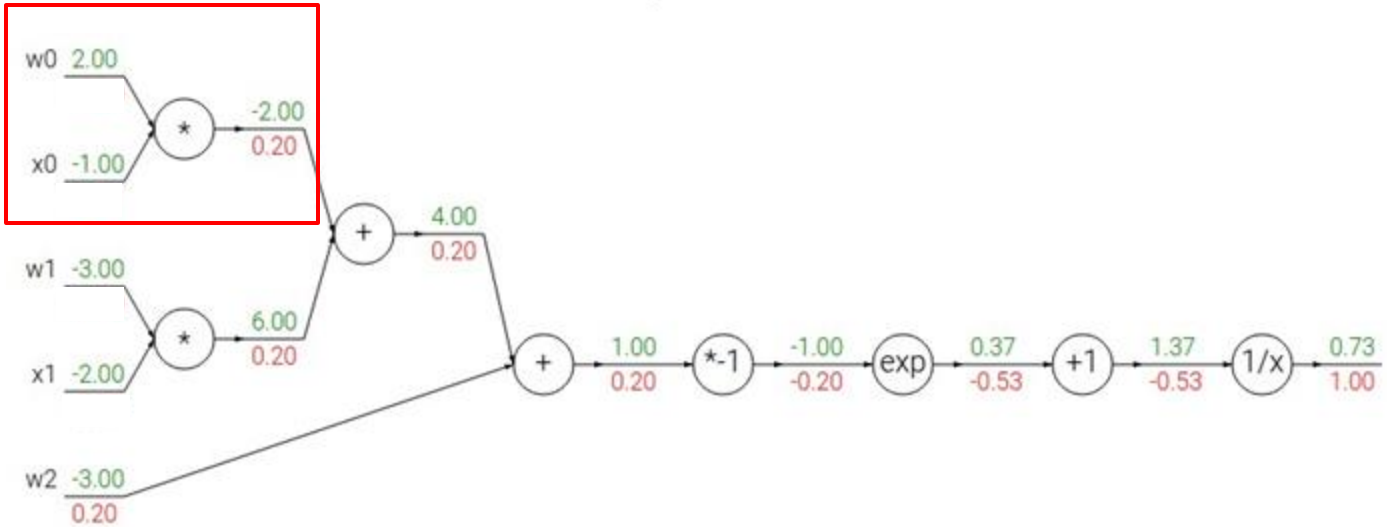
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

A. Alahi

Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

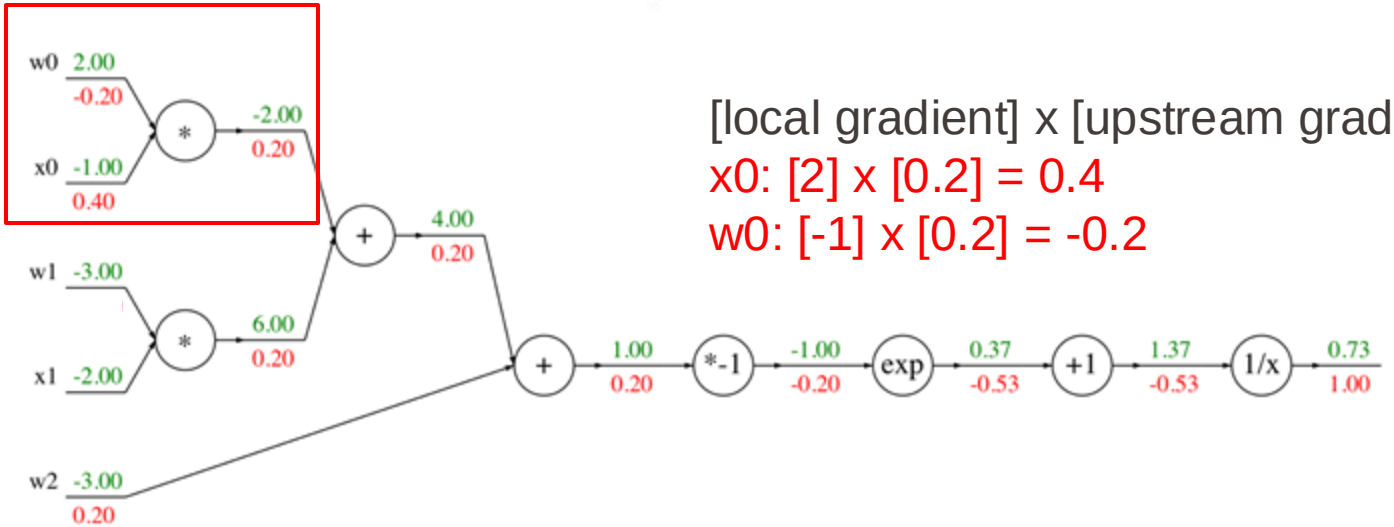
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

A. Alahi

Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



[local gradient] x [upstream gradient]

x0: [2] x [0.2] = 0.4

w0: [-1] x [0.2] = -0.2

$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

A. Alahi

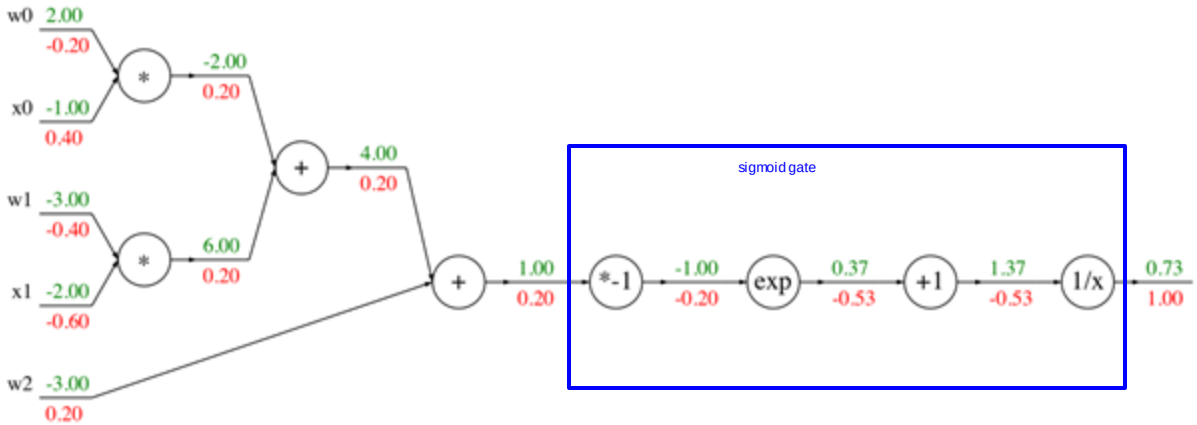
Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

$$\boxed{\sigma(x) = \frac{1}{1+e^{-x}}}$$

sigmoid function

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = (1-\sigma(x))\,\sigma(x)$$



A. Alahi

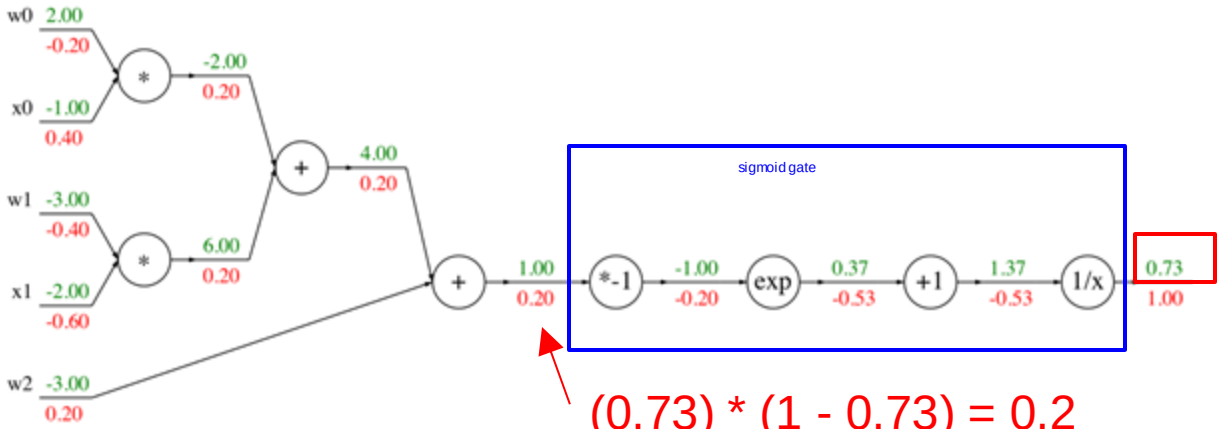
Backprop

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

$$\boxed{\sigma(x) = \frac{1}{1+e^{-x}}}$$

sigmoid function

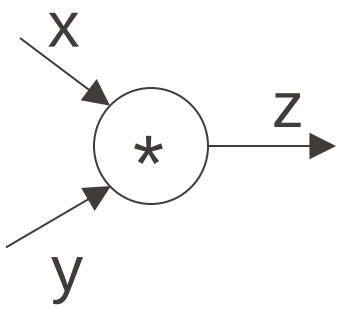
$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = (1-\sigma(x))\,\sigma(x)$$



(0.73) * (1 - 0.73) = 0.2

A. Alahi

Modularized implementation: forward / backward API



(x,y,z are scalars)

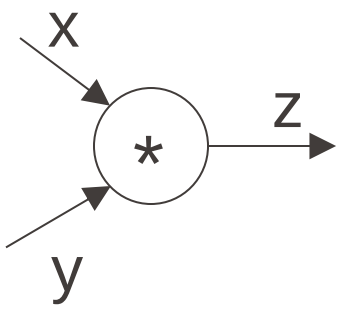
```
class MultiplyGate(object):
    def forward(x,y):
        z = x*y
        return z
    def backward(dz):
        # dx = ... #todo
        # dy = ... #todo
        return [dx, dy]
```

$$\frac{\partial L}{\partial z}$$

$$\frac{\partial L}{\partial x}$$

A. Alahi

Modularized implementation: forward / backward API



(x,y,z are scalars)

```python
class MultiplyGate(object):
    def forward(x,y):
        z = x*y
        self.x = x # must keep these around!
        self.y = y
        return z
    def backward(dz):
        dx = self.y * dz # [dz/dx * dL/dz]
        dy = self.x * dz # [dz/dy * dL/dz]
        return [dx, dy]
```


A. Alahi

Recap on training a neural network…

A. Alahi

Loop:

1. Sample **a batch of data**
2. Forward **pass to get the loss**
3. Backprop **to calculate gradient**
4. Update **parameters using the gradient**

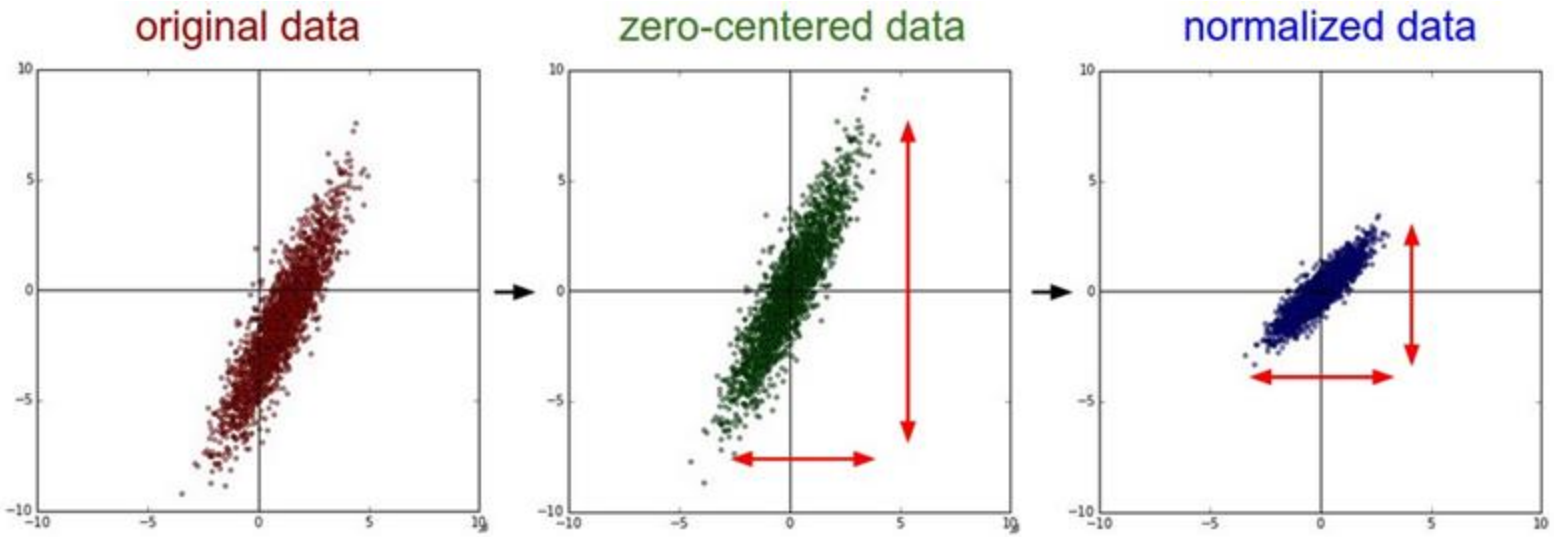
- **B**ackpropagation = recursive application of the chain rule along a computational graph to compute the gradients of all inputs/parameters/intermediates
- **F**orward pass computes result of an operation and save any intermediates needed for gradient computation in memory
- **B**ackward pass applies the chain rule to compute the gradient of the loss function with respect to the inputs

NN TIPS & TRICKS

NN TIPS & TRICKS

- Preprocess data

Data pre-processing




```
X -= np.mean(X, axis = 0)
```

```
X /= np.std(X, axis = 0)
```

For ex, in your lab…

- Subtract per-channel mean (in imageNet it's ~0.5)

(mean along each channel = 3 numbers)

- Divide per-channel STD (in imageNet it's ~0.25)

(STD along each channel = 3 numbers)

A. Alahi

NN TIPS & TRICKS

- Preprocess data
  - **Normalize each dimension of the input**

NN TIPS & TRICKS

- Preprocess data
- Design an architecture
  - Loss function
  - Layers type
  - #layers, #Filters

- **Normalize each dimension of the input**
- **Start from state-of-the-art designs**
  - **Select task appropriate loss function (pytorch losses)**
  - **“Conv” for signals such as images, and more recently Transformers**
  - **Start with popular designs**

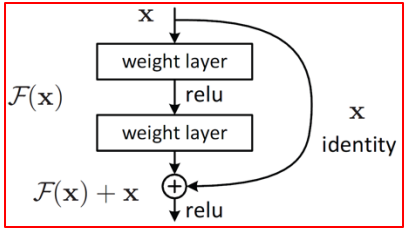
Residual neural networks

- ResNets (He et al., 2015):
Add shortcuts (skip-connections) to jump over some layers

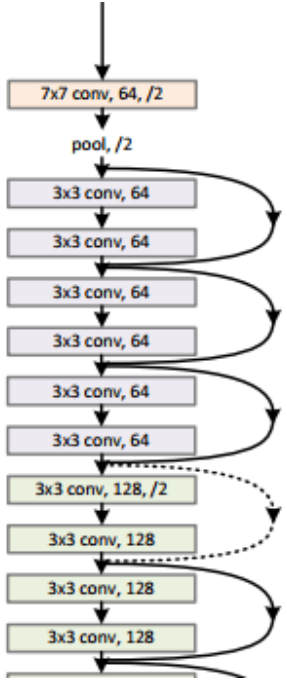
- Deeper models are harder to optimize, and in particular, don't learn identity functions well

- Skip-connections make identity functions easier to learn, helps during training

- ResNets stack **residual blocks** on top of each other to form deep networks (e.g. ResNet-50, ResNet-101, …)



residual blocks



He and al, Deep Residual Learning for Image Recognition, 2015

Design an architecture

- https://playground.tensorflow.org/

NN TIPS & TRICKS

- Preprocess data
- Design an architecture
  - Loss function
  - Layers type
  - #layers, #Filters

- **Normalize each dimension of the input**
- **Start from state-of-the-art designs**
  - **Select task appropriate loss function (pytorch losses)**
  - **“Conv” for signals such as images, and more recently Transformers**
  - **Start with popular designs**

NN TIPS & TRICKS

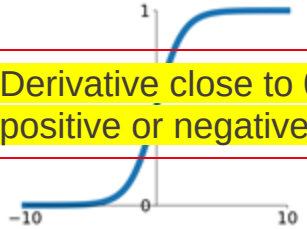
- Preprocess data
- Design an architecture
  - Loss function
  - Layers type
  - #layers, #Filters
  - Activation function

- **Normalize each dimension of the input**
- **Start from state-of-the-art designs**
  - **Select task appropriate loss function (pytorch losses)**
  - **“Conv” for signals such as images, and more recently Transformers**
  - **Start with popular designs**
  - **ReLu, elu**

Activation functions

Sigmoid

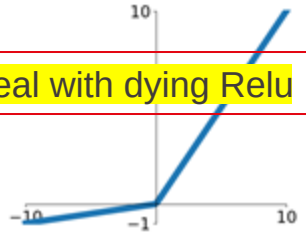
$$\sigma(x) = \frac{1}{1+e^{-x}}$$



Derivative close to 0 for large positive or negative inputs

Leaky ReLU

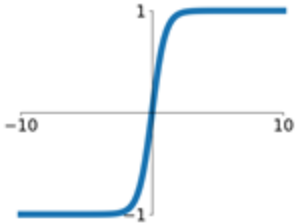
$$\max(0.1x, x)$$



Deal with dying Relu

tanh

$$\tanh(x)$$

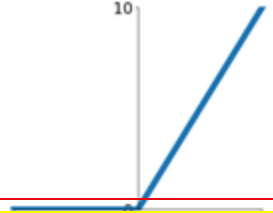


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ReLU

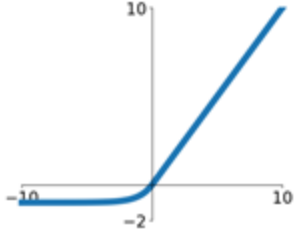
$$\max(0, x)$$



Derivative of output with respect to the input is 1 for inputs great than 0
=> Training more stable and efficient

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



A. Alahi

NN TIPS & TRICKS

- Preprocess data
- Design an architecture
  - Loss function
  - Layers type
  - #layers, #Filters
  - Activation function
- Sanity check
  - Gradient check
  - Loss dynamics
  - Overfit subset of the data

›Normalize each dimension of the input
›Start from state-of-the-art designs
  ›Select task appropriate loss function (pytorch losses)
  ›“Conv” for signals such as images, and more recently Transformers
  ›Start with popular designs
  ›ReLu, elu

NN TIPS & TRICKS

- Preprocess data
  - **Normalize each dimension of the input**
- Design an architecture
  - **Start from state-of-the-art designs**
    - Loss function
      - **Select task appropriate loss function (pytorch losses)**
    - Layers type
      - **"Conv" for signals such as images, and more recently Transformers**
    - #layers, #Filters
      - **Start with popular designs**
    - Activation function
      - **ReLu, elu**
- Sanity check
  - **Only once**
    - Gradient check
      - **Analytical VS num grad (while turning all regularization off) *(fix rand seed)***
    - Loss dynamics
      - **At chance or when regularization is increased**
    - Overfit subset of the data

Double check that the loss is reasonable:

A. Alahi

```
def init_two_layer_model(input_size, hidden_size, output_size):
  # initialize a model
  model = {}
  model['W1'] = 0.0001 * np.random.randn(input_size, hidden_size)
  model['b1'] = np.zeros(hidden_size)
  model['W2'] = 0.0001 * np.random.randn(hidden_size, output_size)
  model['b2'] = np.zeros(output_size)
  return model
```

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
loss, grad = two_layer_net(X_train, model, y_train, 0.0)
print loss
```

disable regularization

```
2.30261216167
```

loss ~2.3.
“correct “ for
10 classes

returns the loss and the
gradient for all parameters

Double check that the loss is reasonable:

A. Alahi

```
def init_two_layer_model(input_size, hidden_size, output_size):
  # initialize a model
  model = {}
  model['W1'] = 0.0001 * np.random.randn(input_size, hidden_size)
  model['b1'] = np.zeros(hidden_size)
  model['W2'] = 0.0001 * np.random.randn(hidden_size, output_size)
  model['b2'] = np.zeros(output_size)
  return model
```

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
loss, grad = two_layer_net(X_train, model, y_train, 1e3)
print loss
```

crank up regularization

```
3.06859716482
```

loss went up, good. (sanity check)

Lets try to train now…

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
X_tiny = X_train[:20] # take 20 examples
y_tiny = y_train[:20]
best_model, stats = trainer.train(X_tiny, y_tiny, X_tiny, y_tiny,
                                  model, two_layer_net,
                                  num_epochs=200, reg=0.0,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = False,
                                  learning_rate=1e-3, verbose=True)
```

Tip: Make sure that you can overfit very small portion of the training data

The above code:

- take the first 20 examples from CIFAR-10
- turn off regularization (reg = 0.0)
- use simple vanilla ‘sgd’

Lets try to train now…

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
X_tiny = X_train[:20] # take 20 examples
y_tiny = y_train[:20]
best_model, stats = trainer.train(X_tiny, y_tiny, X_tiny, y_tiny,
                                  model, two_layer_net,
                                  num_epochs=200, reg=0.0,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = False,
                                  learning_rate=1e-3, verbose=True)
```

Tip: Make sure that you can overfit very small portion of the training data

```
Finished epoch 1 / 200: cost 2.302603, train: 0.400000, val 0.400000, lr 1.000000e-03
Finished epoch 2 / 200: cost 2.302258, train: 0.450000, val 0.450000, lr 1.000000e-03
Finished epoch 3 / 200: cost 2.301849, train: 0.600000, val 0.600000, lr 1.000000e-03
Finished epoch 4 / 200: cost 2.301196, train: 0.650000, val 0.650000, lr 1.000000e-03
Finished epoch 5 / 200: cost 2.300044, train: 0.650000, val 0.650000, lr 1.000000e-03
Finished epoch 6 / 200: cost 2.297864, train: 0.550000, val 0.550000, lr 1.000000e-03
Finished epoch 7 / 200: cost 2.293595, train: 0.600000, val 0.600000, lr 1.000000e-03
Finished epoch 8 / 200: cost 2.285096, train: 0.550000, val 0.550000, lr 1.000000e-03
Finished epoch 9 / 200: cost 2.268094, train: 0.550000, val 0.550000, lr 1.000000e-03
Finished epoch 10 / 200: cost 2.234787, train: 0.500000, val 0.500000, lr 1.000000e-03
Finished epoch 11 / 200: cost 2.173187, train: 0.500000, val 0.500000, lr 1.000000e-03
Finished epoch 12 / 200: cost 2.076862, train: 0.500000, val 0.500000, lr 1.000000e-03
Finished epoch 13 / 200: cost 1.974090, train: 0.400000, val 0.400000, lr 1.000000e-03
Finished epoch 14 / 200: cost 1.895885, train: 0.400000, val 0.400000, lr 1.000000e-03
Finished epoch 15 / 200: cost 1.820876, train: 0.450000, val 0.450000, lr 1.000000e-03
Finished epoch 16 / 200: cost 1.737430, train: 0.450000, val 0.450000, lr 1.000000e-03
Finished epoch 17 / 200: cost 1.642356, train: 0.500000, val 0.500000, lr 1.000000e-03
Finished epoch 18 / 200: cost 1.535239, train: 0.600000, val 0.600000, lr 1.000000e-03
Finished epoch 19 / 200: cost 1.421527, train: 0.600000, val 0.600000, lr 1.000000e-03
```

Very small loss, train accuracy 1.00, nice!

```
Finished epoch 195 / 200: cost 0.002694, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 196 / 200: cost 0.002674, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 197 / 200: cost 0.002655, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 198 / 200: cost 0.002635, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 199 / 200: cost 0.002617, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 200 / 200: cost 0.002597, train: 1.000000, val 1.000000, lr 1.000000e-03
finished optimization. best validation accuracy: 1.000000
```

Lets try to train now…

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
best_model, stats = trainer.train(X_train, y_train, X_val, y_val,
                                  model, two_layer_net,
                                  num_epochs=10, reg=0.000001,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = True,
                                  learning_rate=1e-6, verbose=True)
```

Start with small regularization and find learning rate that makes the loss go down.

Lets try to train now…

Start with small regularization and find learning rate that makes the loss go down.

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
best_model, stats = trainer.train(X_train, y_train, X_val, y_val,
                                  model, two_layer_net,
                                  num_epochs=10, reg=0.000001,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = True,
                                  learning_rate=1e-6, verbose=True)
```

```
Finished epoch 1 / 10: cost 2.302576, train: 0.080000, val 0.103000, lr 1.000000e-06
Finished epoch 2 / 10: cost 2.302582, train: 0.121000, val 0.124000, lr 1.000000e-06
Finished epoch 3 / 10: cost 2.302558, train: 0.119000, val 0.138000, lr 1.000000e-06
Finished epoch 4 / 10: cost 2.302519, train: 0.127000, val 0.151000, lr 1.000000e-06
Finished epoch 5 / 10: cost 2.302517, train: 0.158000, val 0.171000, lr 1.000000e-06
Finished epoch 6 / 10: cost 2.302518, train: 0.179000, val 0.172000, lr 1.000000e-06
Finished epoch 7 / 10: cost 2.302466, train: 0.180000, val 0.176000, lr 1.000000e-06
Finished epoch 8 / 10: cost 2.302452, train: 0.175000, val 0.185000, lr 1.000000e-06
Finished epoch 9 / 10: cost 2.302459, train: 0.206000, val 0.192000, lr 1.000000e-06
Finished epoch 10 / 10: cost 2.302420, train: 0.190000, val 0.192000, lr 1.000000e-06
finished optimization. best validation accuracy: 0.192000
```

Loss barely changing

Lets try to train now…

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
best_model, stats = trainer.train(X_train, y_train, X_val, y_val,
                                  model, two_layer_net,
                                  num_epochs=10, reg=0.000001,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = True,
                                  learning_rate=1e-6, verbose=True)
```

```
Finished epoch 1 / 10: cost 2.302576, train: 0.080000, val 0.103000, lr 1.000000e-06
Finished epoch 2 / 10: cost 2.302582, train: 0.121000, val 0.124000, lr 1.000000e-06
Finished epoch 3 / 10: cost 2.302558, train: 0.119000, val 0.138000, lr 1.000000e-06
Finished epoch 4 / 10: cost 2.302519, train: 0.127000, val 0.151000, lr 1.000000e-06
Finished epoch 5 / 10: cost 2.302517, train: 0.158000, val 0.171000, lr 1.000000e-06
Finished epoch 6 / 10: cost 2.302518, train: 0.179000, val 0.172000, lr 1.000000e-06
Finished epoch 7 / 10: cost 2.302466, train: 0.180000, val 0.176000, lr 1.000000e-06
Finished epoch 8 / 10: cost 2.302452, train: 0.175000, val 0.185000, lr 1.000000e-06
Finished epoch 9 / 10: cost 2.302459, train: 0.206000, val 0.192000, lr 1.000000e-06
Finished epoch 10 / 10: cost 2.302420, train: 0.190000, val 0.192000, lr 1.000000e-06
finished optimization. best validation accuracy: 0.192000
```

Loss barely changing: Learning rate is probably too low

Start with small regularization and find learning rate that makes the loss go down.

- **loss not going down:**

  learning rate too low

Lets try to train now…

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
best_model, stats = trainer.train(X_train, y_train, X_val, y_val,
                                  model, two_layer_net,
                                  num_epochs=10, reg=0.000001,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = True,
                                  learning_rate=1e-6, verbose=True)
```

```
Finished epoch 1 / 10: cost 2.302576, train: 0.080000, val 0.103000, lr 1.000000e-06
Finished epoch 2 / 10: cost 2.302582, train: 0.121000, val 0.124000, lr 1.000000e-06
Finished epoch 3 / 10: cost 2.302558, train: 0.119000, val 0.138000, lr 1.000000e-06
Finished epoch 4 / 10: cost 2.302519, train: 0.127000, val 0.151000, lr 1.000000e-06
Finished epoch 5 / 10: cost 2.302517, train: 0.158000, val 0.171000, lr 1.000000e-06
Finished epoch 6 / 10: cost 2.302518, train: 0.179000, val 0.172000, lr 1.000000e-06
Finished epoch 7 / 10: cost 2.302466, train: 0.180000, val 0.176000, lr 1.000000e-06
Finished epoch 8 / 10: cost 2.302452, train: 0.175000, val 0.185000, lr 1.000000e-06
Finished epoch 9 / 10: cost 2.302459, train: 0.206000, val 0.192000, lr 1.000000e-06
Finished epoch 10 / 10: cost 2.302420, train: 0.190000, val 0.192000, lr 1.000000e-06
finished optimization. best validation accuracy: 0.192000
```

Loss barely changing: Learning rate is probably too low

Start with small regularization and find learning rate that makes the loss go down.

- **loss not going down:**

  learning rate too low

Lets try to train now…

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
best_model, stats = trainer.train(X_train, y_train, X_val, y_val,
                                  model, two_layer_net,
                                  num_epochs=10, reg=0.000001,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = True,
```

Now let's try learning rate 1e6.

Start with small regularization and find learning rate that makes the loss go down.

- **loss not going down:**

  learning rate too low

Lets try to train now...

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
best_model, stats = trainer.train(X_train, y_train, X_val, y_val,
                                  model, two_layer_net,
                                  num_epochs=10, reg=0.000001,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = True,
                                  learning_rate=1e6, verbose=True)
```

```
/home/karpathy/cs231n/code/cs231n/classifiers/neural_net.py:50: RuntimeWarning: divide by zero en
countered in log
  data_loss = -np.sum(np.log(probs[range(N), y])) / N
/home/karpathy/cs231n/code/cs231n/classifiers/neural_net.py:48: RuntimeWarning: invalid value enc
ountered in subtract
  probs = np.exp(scores - np.max(scores, axis=1, keepdims=True))
Finished epoch 1 / 10: cost nan, train: 0.091000, val 0.087000, lr 1.000000e+06
Finished epoch 2 / 10: cost nan, train: 0.095000, val 0.087000, lr 1.000000e+06
Finished epoch 3 / 10: cost nan, train: 0.100000, val 0.087000, lr 1.000000e+06
```

Start with small regularization and find learning rate that makes the loss go down.

- **loss not going down:**

  learning rate too low

- **loss exploding:**

  learning rate too high

cost: NaN almost always means high learning rate...

PS: NaN also occurs because of:
- log (negative number),
- divide by zeros (variable going to zero)

Lets try to train now…

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
best_model, stats = trainer.train(X_train, y_train, X_val, y_val,
                                  model, two_layer_net,
                                  num_epochs=10, reg=0.000001,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = True,
                                  learning_rate=3e-3, verbose=True)
```

```
Finished epoch 1 / 10: cost 2.186654, train: 0.308000, val 0.306000, lr 3.000000e-03
Finished epoch 2 / 10: cost 2.176230, train: 0.330000, val 0.350000, lr 3.000000e-03
Finished epoch 3 / 10: cost 1.942257, train: 0.376000, val 0.352000, lr 3.000000e-03
Finished epoch 4 / 10: cost 1.827868, train: 0.329000, val 0.310000, lr 3.000000e-03
Finished epoch 5 / 10: cost inf, train: 0.128000, val 0.128000, lr 3.000000e-03
Finished epoch 6 / 10: cost inf, train: 0.144000, val 0.147000, lr 3.000000e-03
```

Start with small regularization and find learning rate that makes the loss go down.

3e-3 is still too high. Cost explodes….

- **loss not going down:**

  learning rate too low

- **loss exploding:**

  learning rate too high

=> Rough range for learning rate we should be cross-validating is somewhere [1e-3 … 1e-5]

NN TIPS & TRICKS

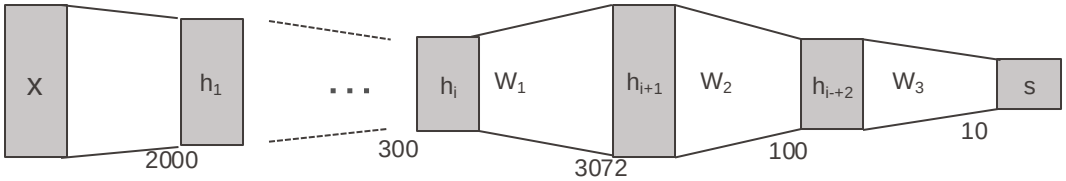
- Preprocess data
  - **Normalize each dimension of the input**
- Design an architecture
  - **Start from state-of-the-art designs**
    - Loss function
      - **Select task appropriate loss function (pytorch losses)**
    - Layers type
      - **"Conv" for signals such as images, and more recently Transformers**
    - #layers, #Filters
      - **Start with popular designs**
    - Activation function
      - **ReLu, elu**
- Sanity check
  - **Only once**
    - Gradient check
      - **Analytical VS num grad (while turning all regularization off) *(fix rand seed)***
    - Loss dynamics
      - **At chance or when regularization is increased**
    - Overfit subset of the data
      - **Use small subset of the data (while regularization is off)**

NN TIPS & TRICKS

- Preprocess data
  - **Normalize each dimension of the input**
- Design an architecture
  - **Start from state-of-the-art designs**
    - Loss function
      - **Select task appropriate loss function (pytorch losses)**
    - Layers type
      - **"Conv" for signals such as images, and more recently Transformers**
    - #layers, #Filters
      - **Start with popular designs**
    - Activation function
      - **ReLu, elu**
- Sanity check
  - **Only once**
    - Gradient check
      - **Analytical VS num grad (while turning all regularization off) *(fix rand seed)***
    - Loss dynamics
      - **At chance or when regularization is increased**
    - Overfit subset of the data
      - **Use small subset of the data (while regularization is off)**
- Initialize weight "smartly"

Weight initialization

- Q: what happens when W=0 init is used?



A. Alahi

Weight initialization

A. Alahi

- **Zero initialization**

```
W = 0.
```

- **Small random numbers**

(gaussian with zero mean and 1e-2 standard deviation)

```
W = 0.01* np.random.randn(D,H)
```

- **Calibrating with number of inputs**

[Glorot et al., 2015] “Xavier init”:

`w = np.random.randn(n) / sqrt(n)`, where `n` is the number of its inputs.

[He et al., 2015]:

`w = np.random.randn(n) * sqrt(2.0/n)`,

NN TIPS & TRICKS

- Preprocess data — **Normalize each dimension of the input**
- Design an architecture — **Start from state-of-the-art designs**
  - Loss function — **Select task appropriate loss function (pytorch losses)**
  - Layers type — **"Conv" for signals such as images, and more recently Transformers**
  - #layers, #Filters — **Start with popular designs**
  - Activation function — **ReLu, elu**
- Sanity check — **Only once**
  - Gradient check — **Analytical VS num grad (while turning all regularization off) *(fix rand seed)***
  - Loss dynamics — **At chance or when regularization is increased**
  - Overfit subset of the data — **Use small subset of the data (while regularization is off)**
- Initialize weight "smartly" — **Pytorch/Tensorflow initialization**

Initialize weight "smartly"

- https://www.deeplearning.ai/ai-notes/initialization/index.html

NN TIPS & TRICKS

- Preprocess data
  - **Normalize each dimension of the input**
- Design an architecture
  - **Start from state-of-the-art designs**
    - Loss function
      - **Select task appropriate loss function (pytorch losses)**
    - Layers type
      - **"Conv" for signals such as images, and more recently Transformers**
    - #layers, #Filters
      - **Start with popular designs**
    - Activation function
      - **ReLu, elu**
- Sanity check
  - **Only once**
    - Gradient check
      - **Analytical VS num grad (while turning all regularization off) *(fix rand seed)***
    - Loss dynamics
      - **At chance or when regularization is increased**
    - Overfit subset of the data
      - **Use small subset of the data (while regularization is off)**
- Initialize weight "smartly"
  - **Pytorch/Tensorflow initialization**
- Batch normalization

Batch Normalization

A. Alahi

"you want unit gaussian activations? just make them so."

consider a batch of activations at some layer.
To make each dimension unit gaussian, apply:

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

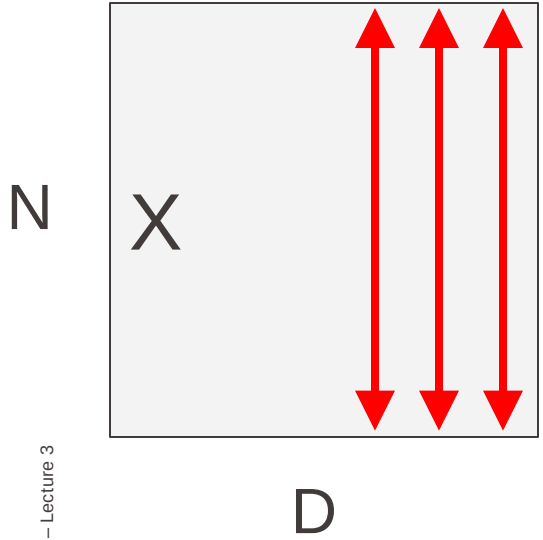
this is a vanilla differentiable function...

[Ioffe and Szegedy, 2015]

Batch Normalization

A. Alahi

"you want unit gaussian activations?
just make them so."



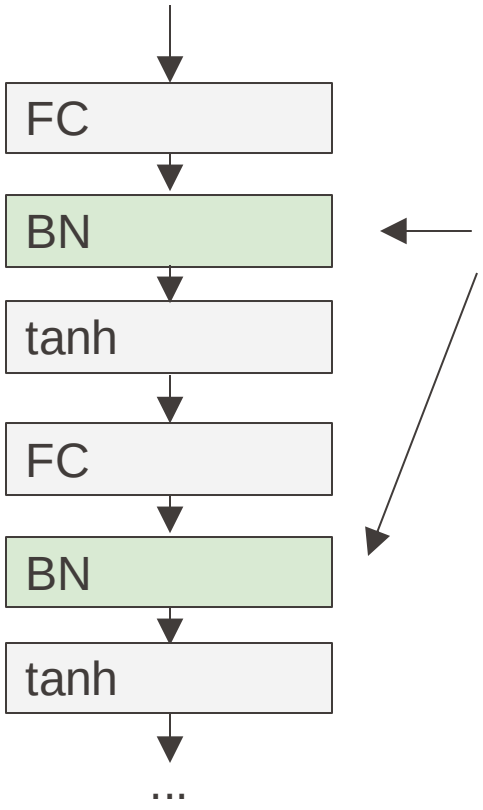
1. compute the empirical mean and variance independently for each dimension.

2. Normalize

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

[Ioffe and Szegedy, 2015]

Batch Normalization



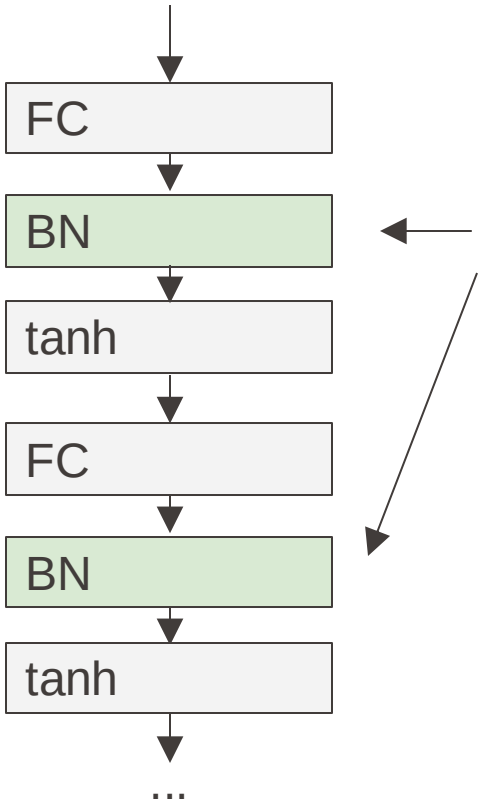
Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

[Ioffe and Szegedy, 2015]

A. Alahi

Batch Normalization



Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

[Ioffe and Szegedy, 2015]

A. Alahi

Batch Normalization

A. Alahi

Normalize:

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

And then allow the network to squash the range if it wants to:

$$y^{(k)} = \gamma^{(k)}\widehat{x}^{(k)} + \beta^{(k)}$$

Note, the network can learn:

$$\gamma^{(k)} = \sqrt{\mathrm{Var}[x^{(k)}]}$$

$$\beta^{(k)} = \mathrm{E}[x^{(k)}]$$

to recover the identity mapping.

[Ioffe and Szegedy, 2015]

Batch Normalization

A. Alahi

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\ldots m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_\mathcal{B} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma_\mathcal{B}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_\mathcal{B})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_\mathcal{B}}{\sqrt{\sigma_\mathcal{B}^2 + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

- Improves gradient flow through the network
- Allows higher learning rates
- Reduces the strong dependence on initialization
- Acts as a form of regularization in a funny way, and slightly reduces the need for dropout, maybe

[Ioffe and Szegedy, 2015]

Batch Normalization

A. Alahi

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\ldots m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_\mathcal{B} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma_\mathcal{B}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_\mathcal{B})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_\mathcal{B}}{\sqrt{\sigma_\mathcal{B}^2 + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Note: at test time BatchNorm layer functions differently:

The mean/std are not computed based on the batch. Instead, a single fixed empirical mean of activations during training is used.

(e.g., can be estimated during training with running averages)

[Ioffe and Szegedy, 2015]

Batch Normalization

- Lubana, E. S., Dick, R., & Tanaka, H. (2021). Beyond BatchNorm: Towards a unified understanding of normalization in deep learning. Neural Information Processing Systems

NN TIPS & TRICKS

- Preprocess data — **Normalize each dimension of the input**
- Design an architecture — **Start from state-of-the-art designs**
  - Loss function — **Select task appropriate loss function (pytorch losses)**
  - Layers type — **"Conv" for signals such as images, and more recently Transformers**
  - #layers, #Filters — **Start with popular designs**
  - Activation function — **ReLu, elu**
- Sanity check — **Only once**
  - Gradient check — **Analytical VS num grad (while turning all regularization off) *(fix rand seed)***
  - Loss dynamics — **At chance or when regularization is increased**
  - Overfit subset of the data — **Use small subset of the data (while regularization is off)**
- Initialize weight "smartly" — **Pytorch/Tensorflow initialization**
- Batch normalization — **Try with and without. Test also Layer norm, group norm**

NN TIPS & TRICKS

- Preprocess data — **Normalize each dimension of the input**
- Design an architecture — **Start from state-of-the-art designs**
  - Loss function — **Select task appropriate loss function (pytorch losses)**
  - Layers type — **"Conv" for signals such as images, and more recently Transformers**
  - #layers, #Filters — **Start with popular designs**
  - Activation function — **ReLu, elu**
- Sanity check — **Only once**
  - Gradient check — **Analytical VS num grad (while turning all regularization off) *(fix rand seed)***
  - Loss dynamics — **At chance or when regularization is increased**
  - Overfit subset of the data — **Use small subset of the data (while regularization is off)**
- Initialize weight "smartly" — **Pytorch/Tensorflow initialization**
- Batch normalization — **Try with and without. Test also Layer norm, group norm**
- Regularization strategies — **Data augment (Augmix, RandAugment, MixUp), no dropout in ResNet, DropBlock**

NN TIPS & TRICKS

- Preprocess data
  - Normalize each dimension of the input
- Design an architecture
  - Start from state-of-the-art designs
    - Loss function
      - Select task appropriate loss function (pytorch losses)
    - Layers type
      - "Conv" for signals such as images, and more recently Transformers
    - #layers, #Filters
      - Start with popular designs
    - Activation function
      - ReLu, elu
- Sanity check
  - Only once
    - Gradient check
      - Analytical VS num grad (while turning all regularization off) *(fix rand seed)*
    - Loss dynamics
      - At chance or when regularization is increased
    - Overfit subset of the data
      - Use small subset of the data (while regularization is off)
- Initialize weight "smartly"
  - Pytorch/Tensorflow initialization
- Batch normalization
  - Try with and without. Test also Layer norm, group norm
- Regularization strategies
  - Data augment (Augmix, RandAugment, MixUp), no dropout in ResNet, DropBlock
- Optimization strategy
  - AdamW, Adam, SGD+Nesterov momentum, warm-up, learning rate scheduler, RAdam

Optimization review

- https://www.deeplearning.ai/ai-notes/optimization/index.html
- On gradient based optimization methods:

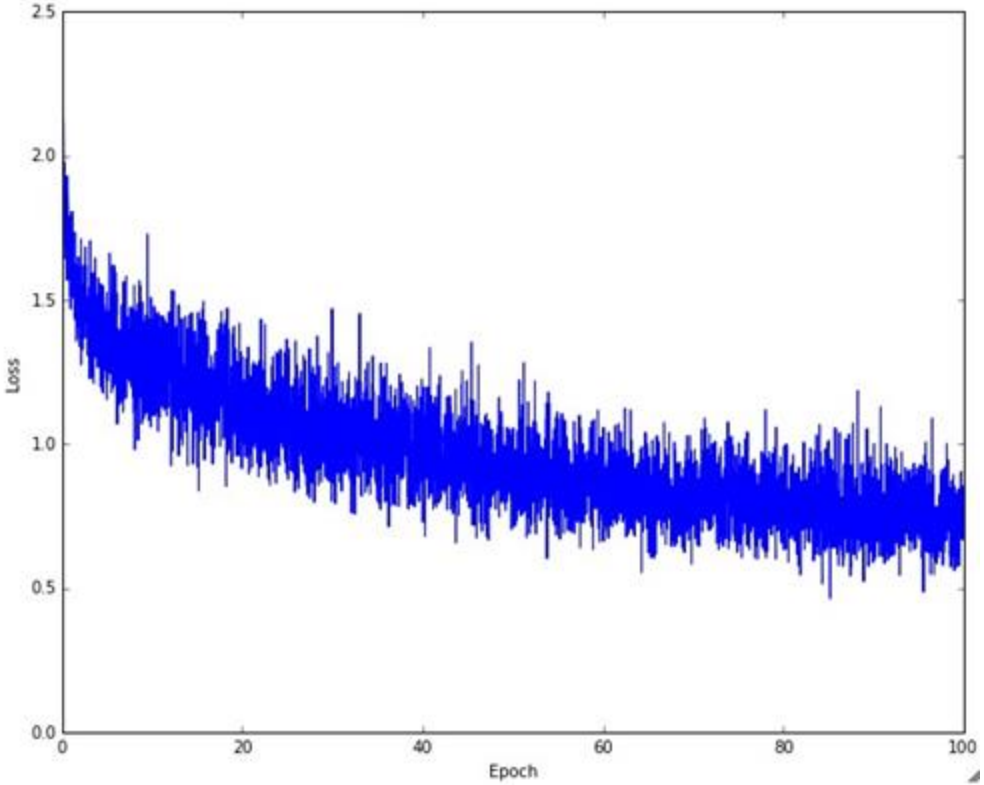
http://ruder.io/optimizing-gradient-descent/

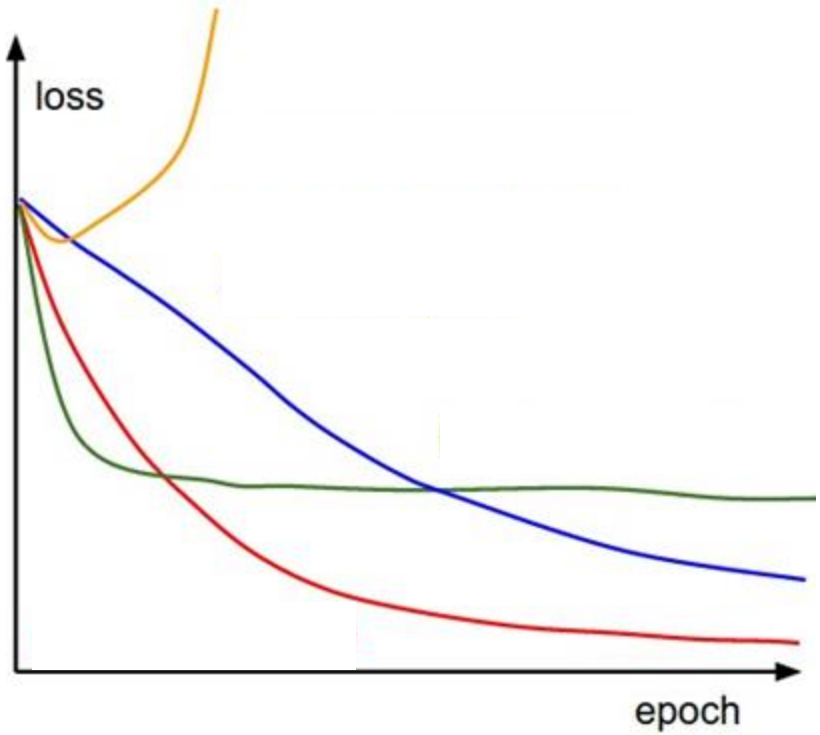
- Bottou, L., Curtis, F. E., & Nocedal, J. (2018). Optimization methods for large-scale machine learning. SIAM Review
- Sun, R.-Y. (2020). Optimization for deep learning: An overview. Journal of the Operations Research Society of China

NN TIPS & TRICKS

- Preprocess data
  - Normalize each dimension of the input
- Design an architecture
  - Start from state-of-the-art designs
    - Loss function
      - Select task appropriate loss function (pytorch losses)
    - Layers type
      - "Conv" for signals such as images, and more recently Transformers
    - #layers, #Filters
      - Start with popular designs
    - Activation function
      - ReLu, elu
- Sanity check
  - Only once
    - Gradient check
      - Analytical VS num grad (while turning all regularization off) *(fix rand seed)*
    - Loss dynamics
      - At chance or when regularization is increased
    - Overfit subset of the data
      - Use small subset of the data (while regularization is off)
- Initialize weight "smartly"
  - Pytorch/Tensorflow initialization
- Batch normalization
  - Try with and without. Test also Layer norm, group norm
- Regularization strategies
  - Data augment (Augmix, RandAugment, MixUp), no dropout in ResNet, DropBlock
- Optimization strategy
  - AdamW, Adam, SGD+Nesterov momentum, warm-up, learning rate scheduler
- Monitor learning process

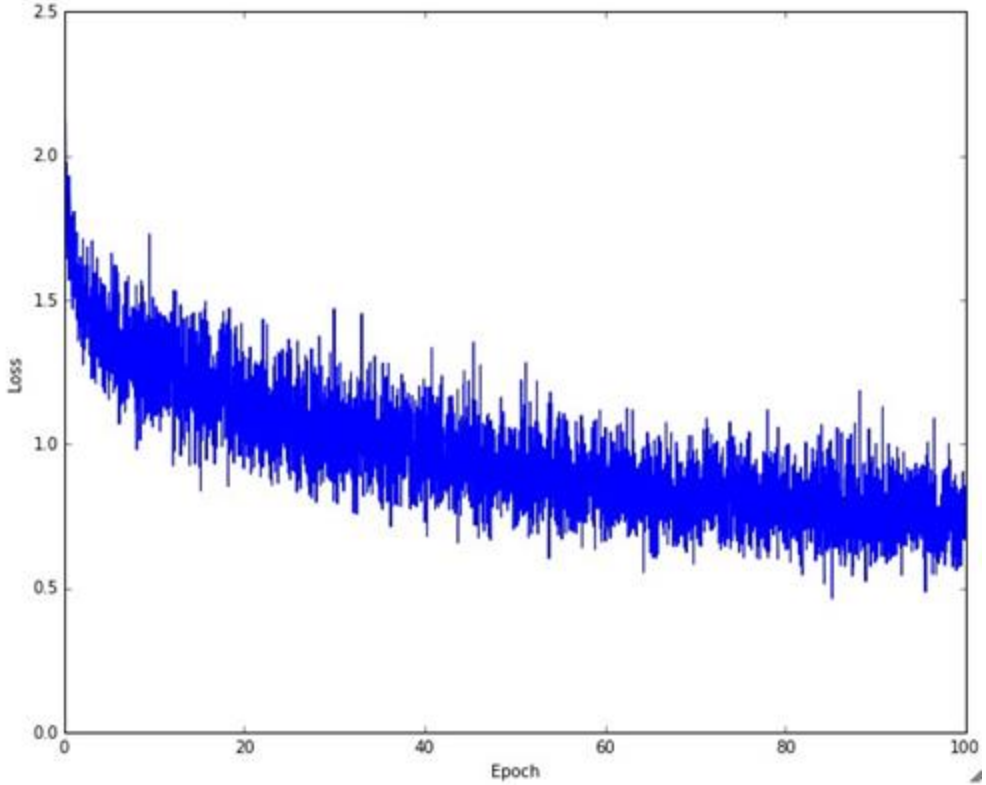
Monitor and visualize the loss curve

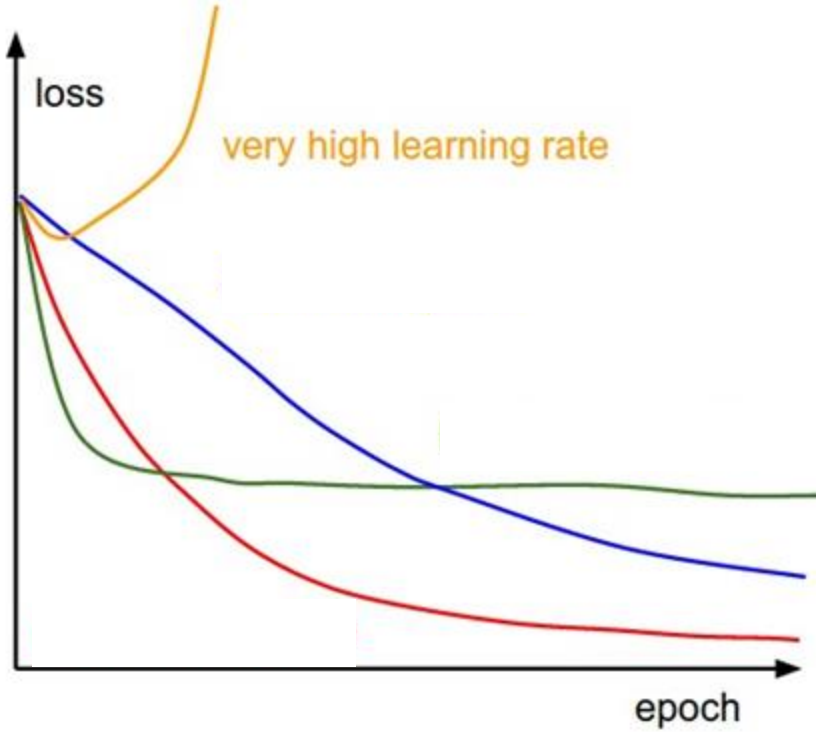




A. Alahi

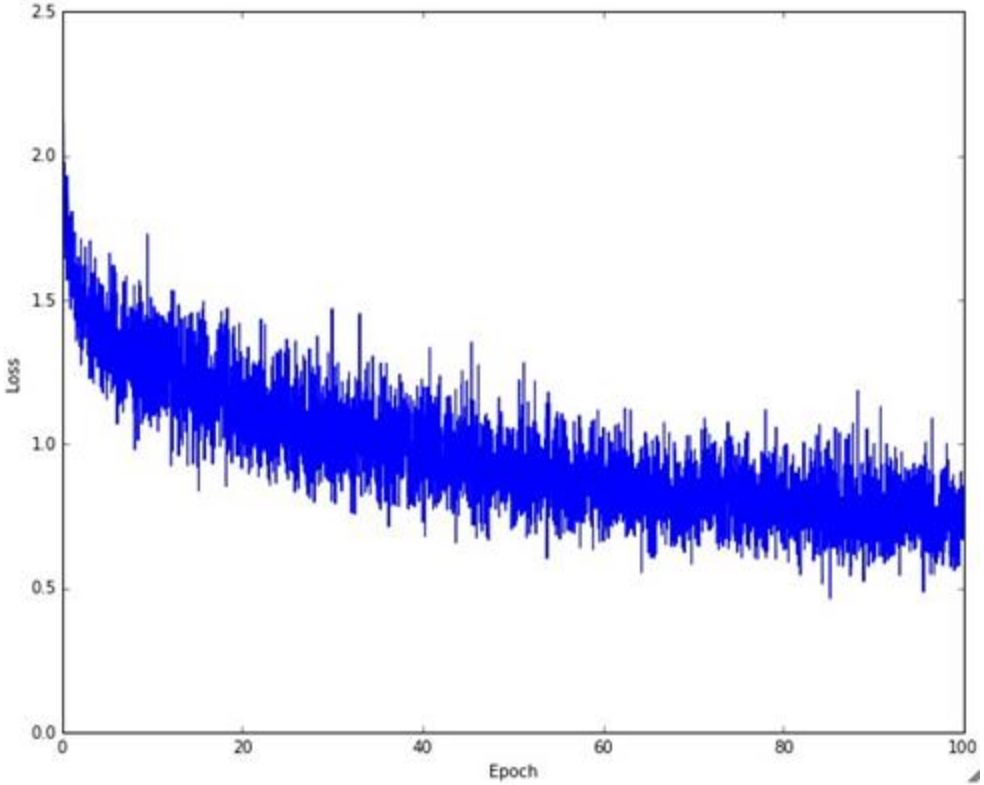
Monitor and visualize the loss curve

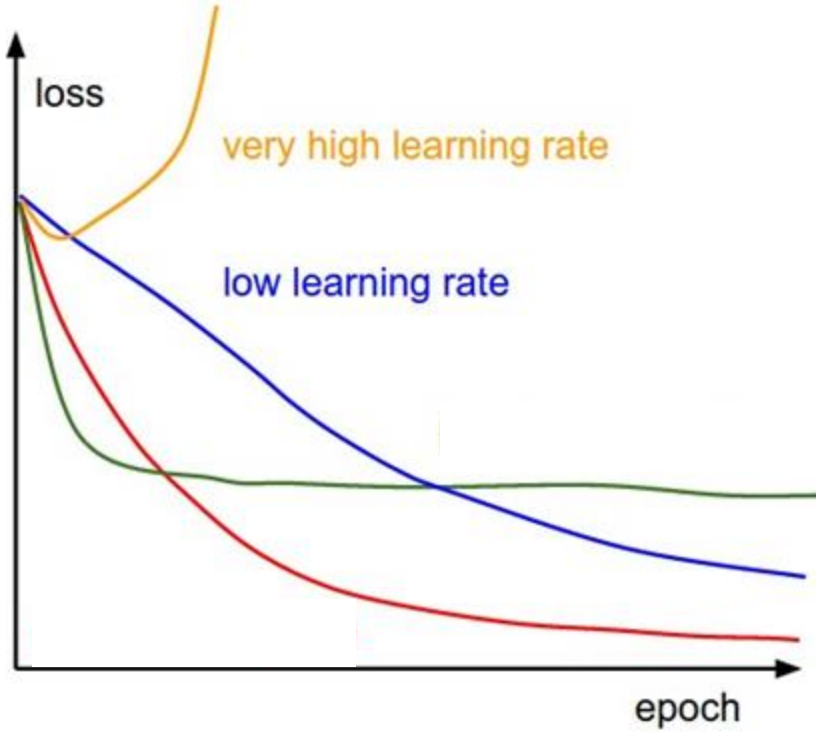




A. Alahi

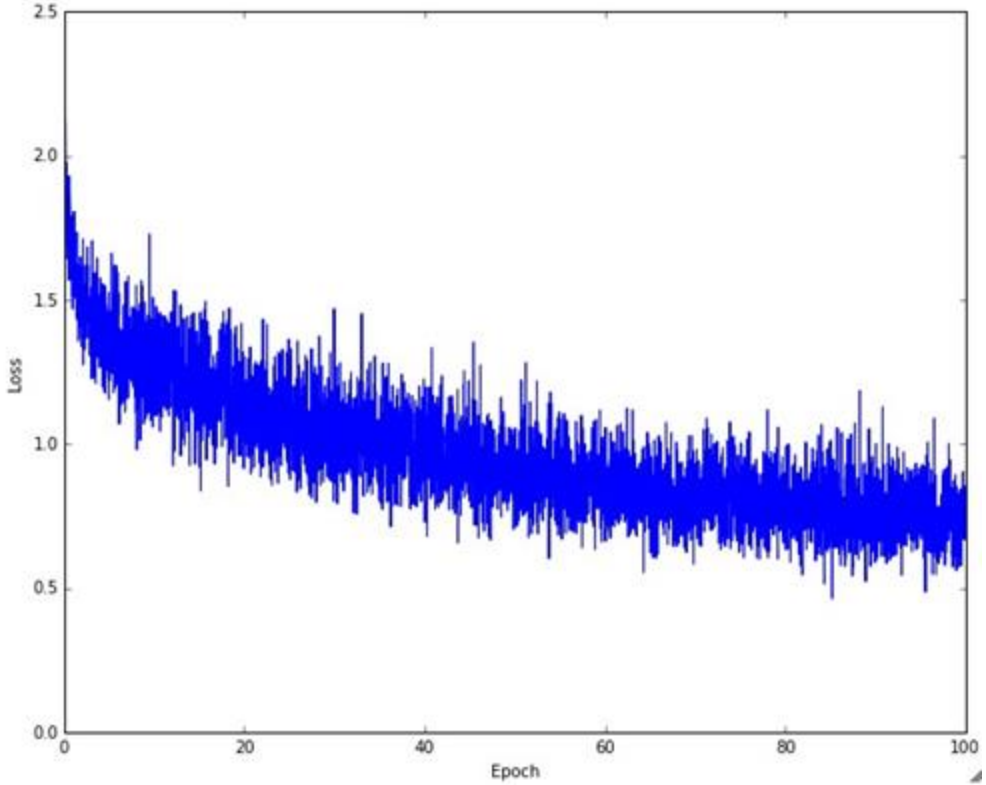
Monitor and visualize the loss curve

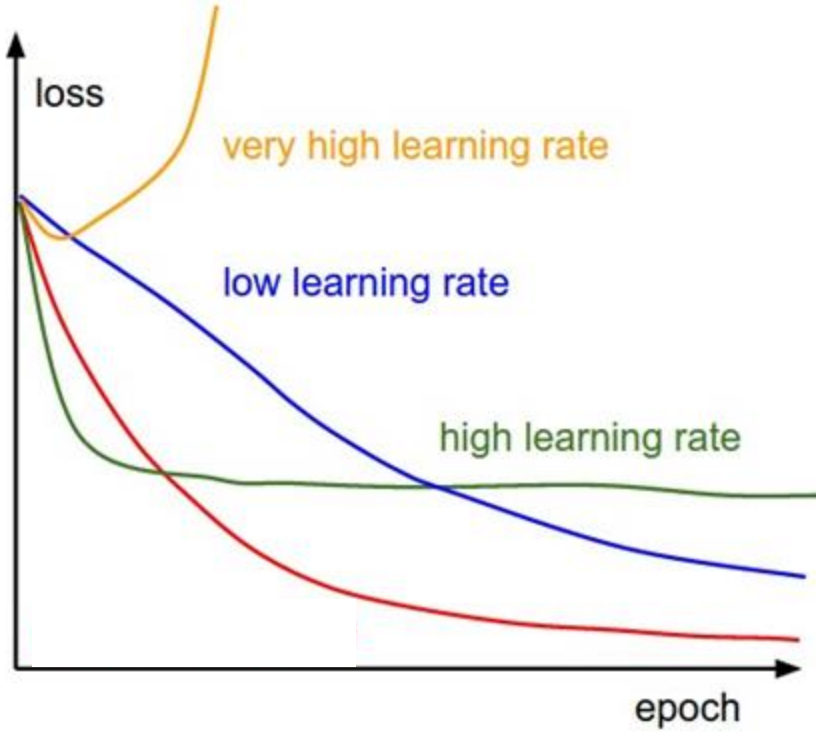




A. Alahi

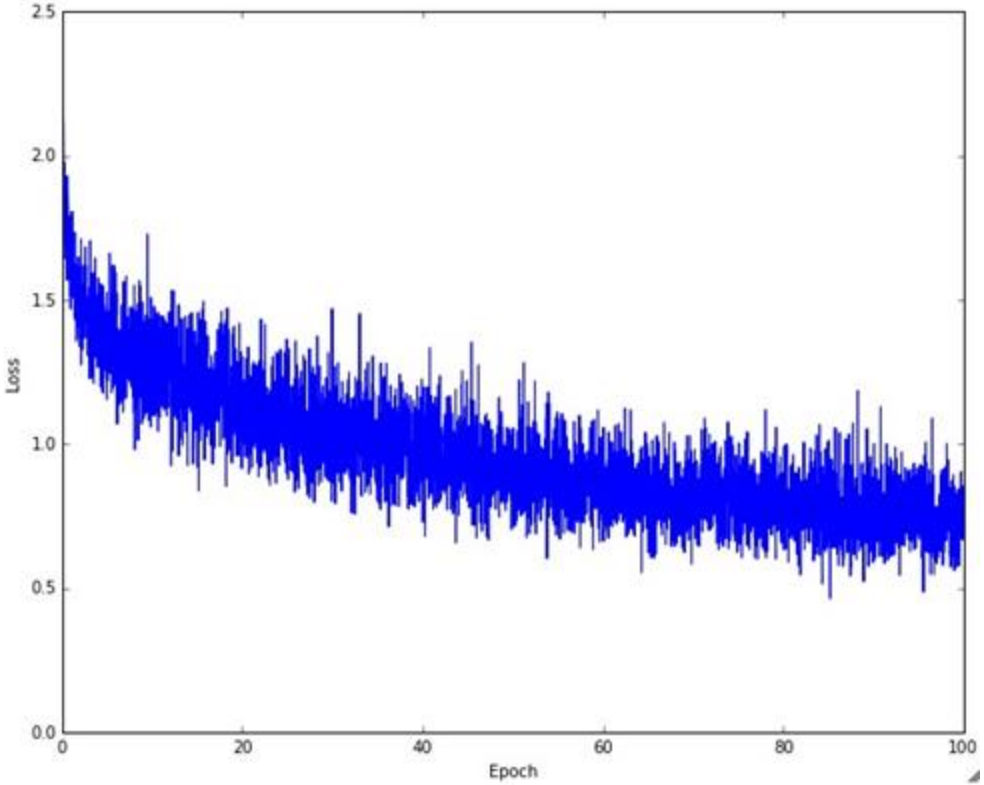
Monitor and visualize the loss curve

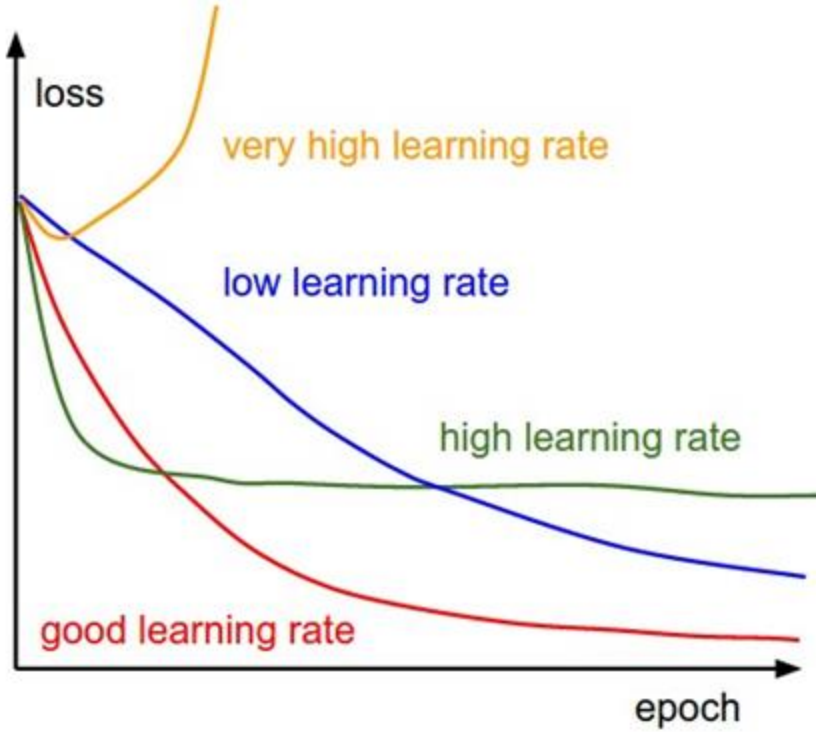




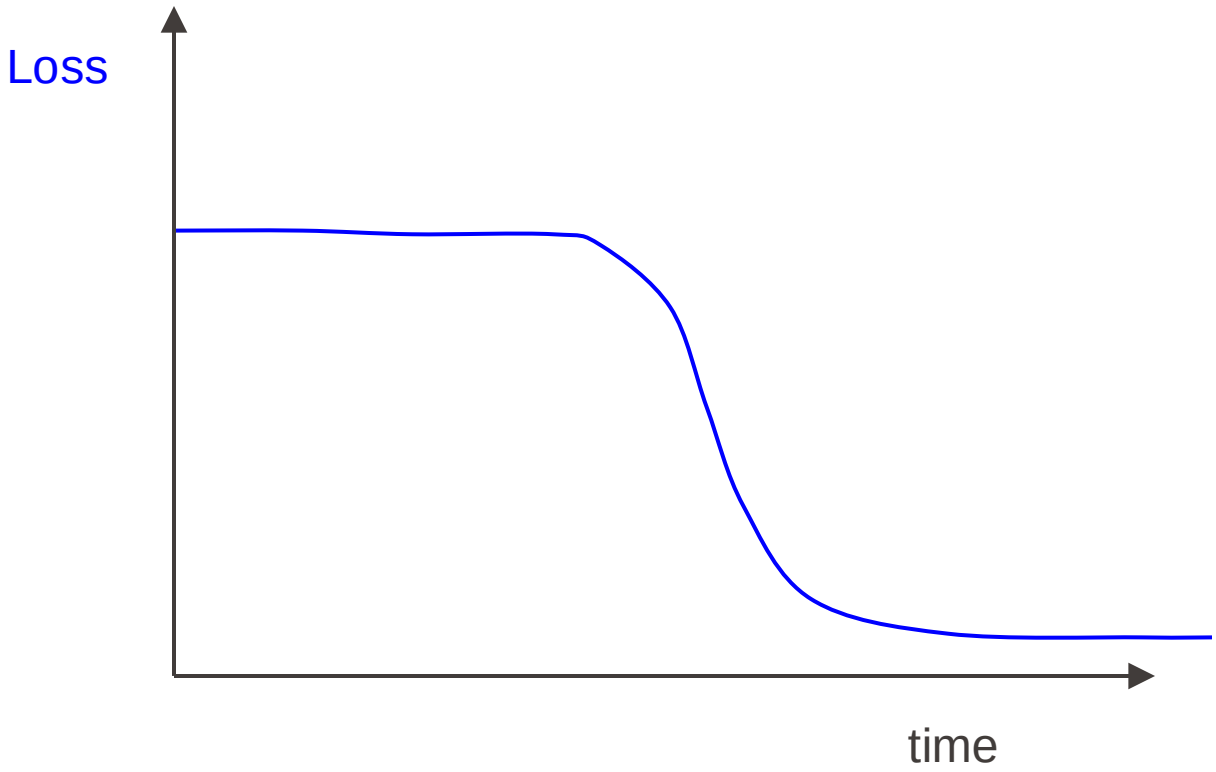
A. Alahi

Monitor and visualize the loss curve

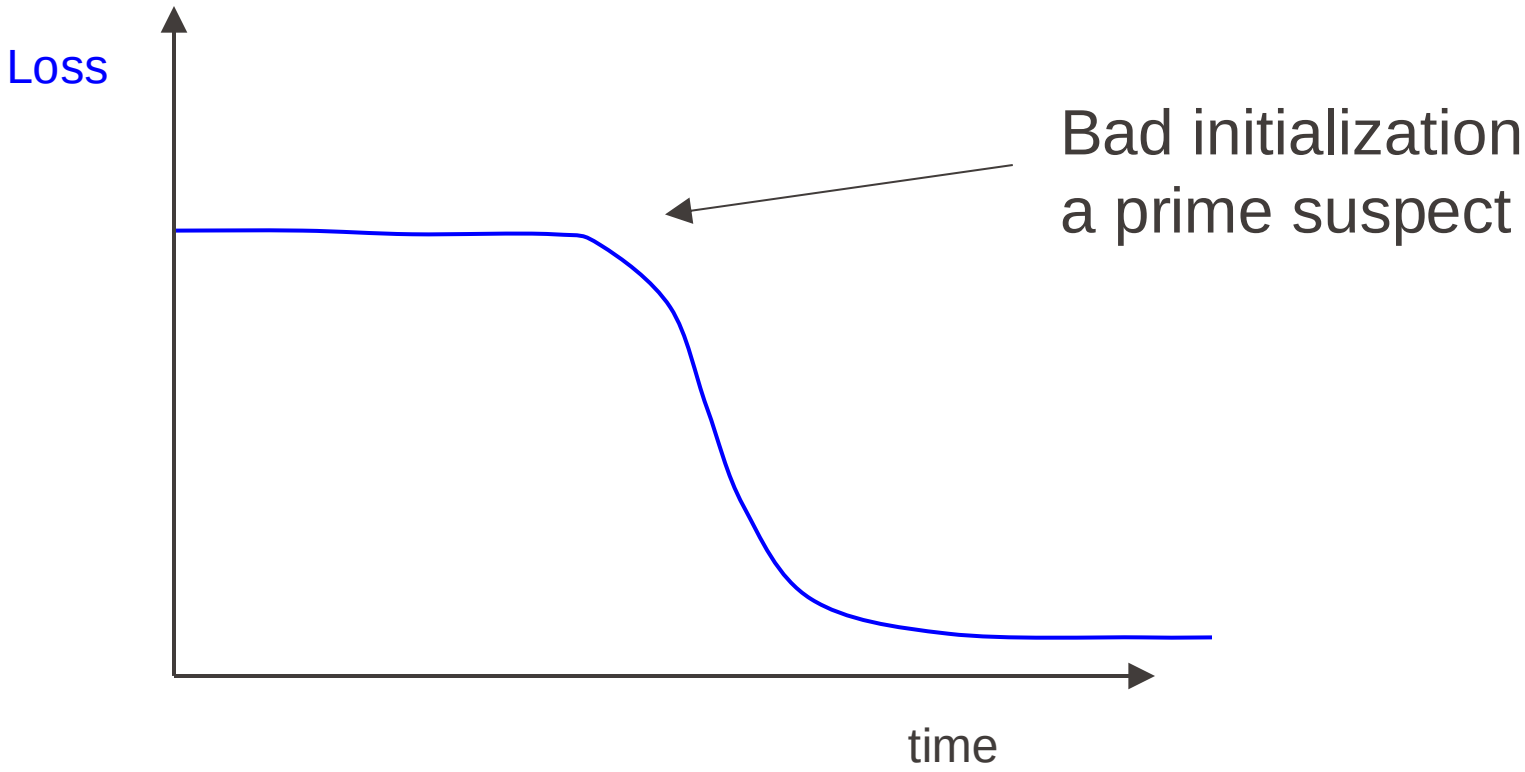




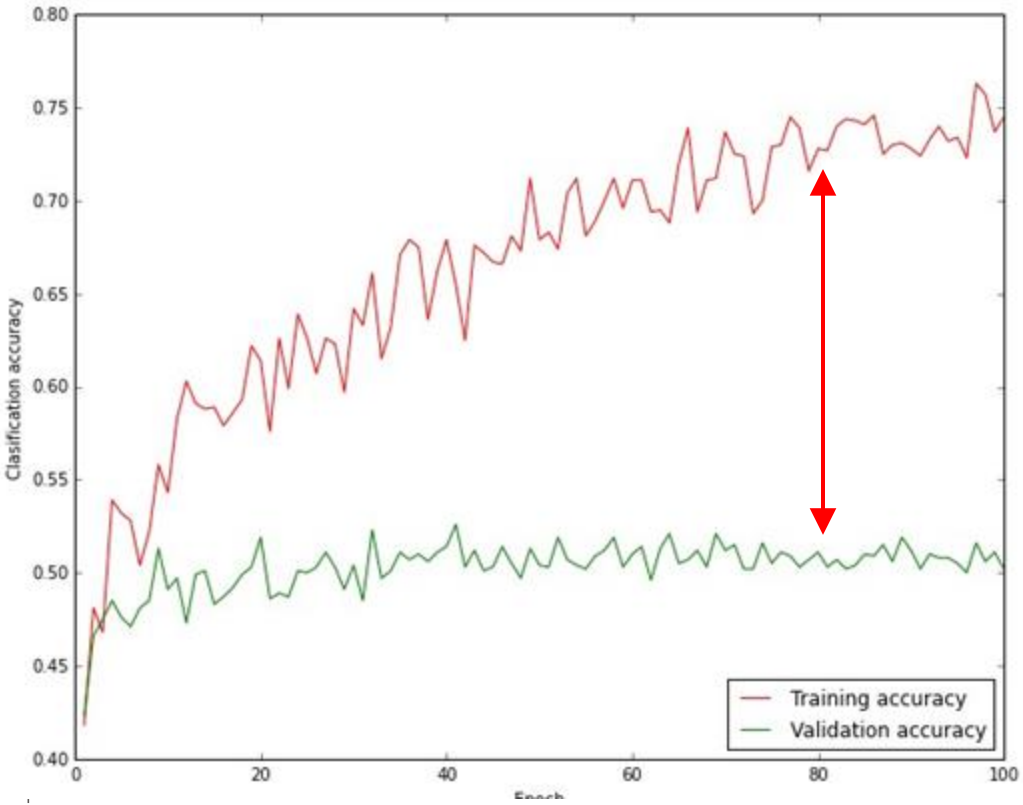
A. Alahi



A. Alahi


Loss
Bad initialization
a prime suspect
time

Monitor and visualize the accuracy:



big gap = overfitting

=> increase regularization strength?

no gap

=> increase model capacity?

A. Alahi

Track the ratio of weight updates / weight magnitudes:

A. Alahi

```
# assume parameter vector W and its gradient vector dW
param_scale = np.linalg.norm(W.ravel())
update = -learning_rate*dW # simple SGD update
update_scale = np.linalg.norm(update.ravel())
W += update # the actual update
print update_scale / param_scale # want ~1e-3
```

Ratio between the updates and values: ~ 0.0002 / 0.02 = 0.01 (about okay)

want this to be somewhere around 0.001 or so

NN TIPS & TRICKS

A. Alahi

- Preprocess data
  - Normalize each dimension of the input
- Design an architecture
  - Start from state-of-the-art designs
    - Loss function
      - Select task appropriate loss function (pytorch losses)
    - Layers type
      - “Conv” for signals such as images, and more recently Transformers
    - #layers, #Filters
      - Start with popular designs
    - Activation function
      - ReLu, elu
- Sanity check
  - Only once
    - Gradient check
      - Analytical VS num grad (while turning all regularization off) *(fix rand seed)*
    - Loss dynamics
      - At chance or when regularization is increased
    - Overfit subset of the data
      - Use small subset of the data (while regularization is off)
- Initialize weight “smartly”
  - Pytorch/Tensorflow initialization
- Batch normalization
  - Try with and without. Test also Layer norm, group norm
- Regularization strategies
  - Data augment (Augmix, RandAugment, MixUp), no dropout in ResNet, DropBlock
- Optimization strategy
  - AdamW, Adam, SGD+Nesterov momentum, warm-up, learning rate scheduler
- Monitor learning process
  - Check loss, training/validation accuracy, ratio of weights updates (e.g., Wandb)

NN TIPS & TRICKS

- Preprocess data
  - Normalize each dimension of the input
- Design an architecture
  - Start from state-of-the-art designs
    - Loss function
      - Select task appropriate loss function (pytorch losses)
    - Layers type
      - “Conv” for signals such as images, and more recently Transformers
    - #layers, #Filters
      - Start with popular designs
    - Activation function
      - ReLu, elu
- Sanity check
  - Only once
    - Gradient check
      - Analytical VS num grad (while turning all regularization off) *(fix rand seed)*
    - Loss dynamics
      - At chance or when regularization is increased
    - Overfit subset of the data
      - Use small subset of the data (while regularization is off)
- Initialize weight “smartly”
  - Pytorch/Tensorflow initialization
- Batch normalization
  - Try with and without. Test also Layer norm, group norm
- Regularization strategies
  - Data augment (Augmix, RandAugment, MixUp), no dropout in ResNet, DropBlock
- Optimization strategy
  - AdamW, Adam, SGD+Nesterov momentum, warm-up, learning rate scheduler
- Monitor learning process
  - Check loss, training/validation accuracy, ratio of weights updates (e.g., Wandb)
- Hyperparameters search

Cross-validation strategy

A. Alahi

- Coarse -> fine cross-validation in stages
- First stage: only a few epochs to get rough idea of what params work
  Second stage: longer running time, finer search
  … (repeat as necessary)

- Tip for detecting explosions in the solver:

If the cost is ever > 3 * original cost, break out early

For example: run coarse search for 5 epochs

```
max_count = 100
for count in xrange(max_count):
      reg = 10**uniform(-5, 5)
      lr = 10**uniform(-3, -6)

      trainer = ClassifierTrainer()
      model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
      trainer = ClassifierTrainer()
      best_model_local, stats = trainer.train(X_train, y_train, X_val, y_val,
                                              model, two_layer_net,
                                              num_epochs=5, reg=reg,
                                              update='momentum', learning_rate_decay=0.9,
                                              sample_batches = True, batch_size = 100,
                                              learning_rate=lr, verbose=False)
```

note it's best to optimize in log space!

```
val_acc: 0.412000, lr: 1.405206e-04, reg: 4.793564e-01, (1 / 100)
val_acc: 0.214000, lr: 7.231888e-06, reg: 2.321281e-04, (2 / 100)
val_acc: 0.208000, lr: 2.119571e-06, reg: 8.011857e+01, (3 / 100)
val_acc: 0.196000, lr: 1.551131e-05, reg: 4.374936e-05, (4 / 100)
val_acc: 0.079000, lr: 1.753300e-05, reg: 1.200424e+03, (5 / 100)
val_acc: 0.223000, lr: 4.215128e-05, reg: 4.196174e+01, (6 / 100)
val_acc: 0.441000, lr: 1.750259e-04, reg: 2.110807e-04, (7 / 100)
val_acc: 0.241000, lr: 6.749231e-05, reg: 4.226413e+01, (8 / 100)
val_acc: 0.482000, lr: 4.296863e-04, reg: 6.642555e-01, (9 / 100)
val_acc: 0.079000, lr: 5.401602e-06, reg: 1.599828e+04, (10 / 100)
val_acc: 0.154000, lr: 1.618508e-06, reg: 4.925252e-01, (11 / 100)
```

nice

A. Alahi

Now run finer search...

```
max_count = 100
for count in xrange(max_count):
      reg = 10**uniform(-5, 5)
      lr = 10**uniform(-3, -6)
```

adjust range →

```
max_count = 100
for count in xrange(max_count):
      reg = 10**uniform(-4, 0)
      lr = 10**uniform(-3, -4)
```

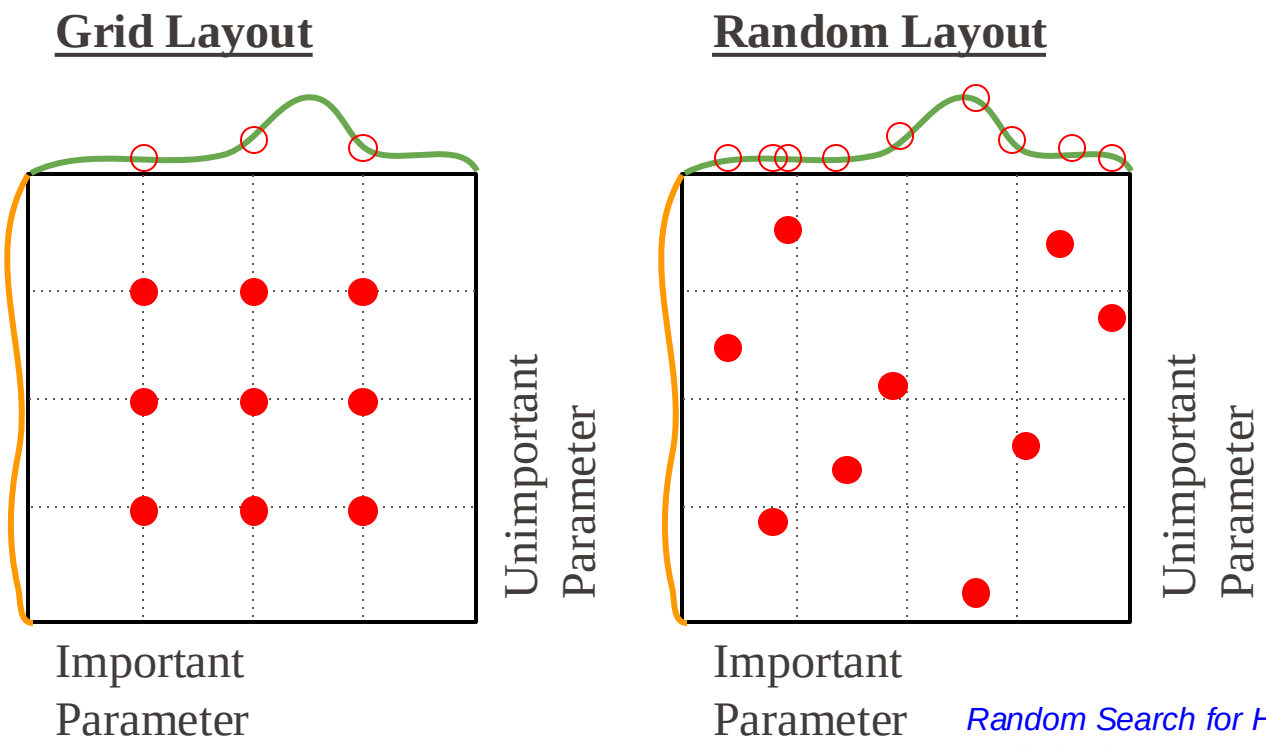
```
val_acc: 0.527000, lr: 5.340517e-04, reg: 4.097824e-01, (0 / 100)
val_acc: 0.492000, lr: 2.279484e-04, reg: 9.991345e-04, (1 / 100)
val_acc: 0.512000, lr: 8.680827e-04, reg: 1.349727e-02, (2 / 100)
val_acc: 0.461000, lr: 1.028377e-04, reg: 1.220193e-02, (3 / 100)
val_acc: 0.460000, lr: 1.113730e-04, reg: 5.244309e-02, (4 / 100)
val_acc: 0.498000, lr: 9.477776e-04, reg: 2.001293e-03, (5 / 100)
val_acc: 0.469000, lr: 1.484369e-04, reg: 4.328313e-01, (6 / 100)
val_acc: 0.522000, lr: 5.586261e-04, reg: 2.312685e-04, (7 / 100)
val_acc: 0.530000, lr: 5.808183e-04, reg: 8.259964e-02, (8 / 100)
val_acc: 0.489000, lr: 1.979168e-04, reg: 1.010889e-04, (9 / 100)
val_acc: 0.490000, lr: 2.036031e-04, reg: 2.406271e-03, (10 / 100)
val_acc: 0.475000, lr: 2.021162e-04, reg: 2.287807e-01, (11 / 100)
val_acc: 0.460000, lr: 1.135527e-04, reg: 3.905040e-02, (12 / 100)
val_acc: 0.515000, lr: 6.947668e-04, reg: 1.562808e-02, (13 / 100)
val_acc: 0.531000, lr: 9.471549e-04, reg: 1.433895e-03, (14 / 100)
val_acc: 0.509000, lr: 3.140888e-04, reg: 2.857518e-01, (15 / 100)
val_acc: 0.514000, lr: 6.438349e-04, reg: 3.033781e-01, (16 / 100)
val_acc: 0.502000, lr: 3.921784e-04, reg: 2.707126e-04, (17 / 100)
val_acc: 0.509000, lr: 9.752279e-04, reg: 2.850865e-03, (18 / 100)
val_acc: 0.500000, lr: 2.412048e-04, reg: 4.997821e-04, (19 / 100)
val_acc: 0.466000, lr: 1.319314e-04, reg: 1.189915e-02, (20 / 100)
val_acc: 0.516000, lr: 8.039527e-04, reg: 1.528291e-02, (21 / 100)
```

53% - relatively good for a 2-layer neural net with 50 hidden neurons.

A. Alahi

Random Search vs. Grid Search

A. Alahi



Random Search for Hyper-Parameter Optimization
Bergstra and Bengio, 2012

Hyperparameters to play with

- network architecture
- learning rate, its decay schedule, update type
- regularization (L2/Dropout strength)

A. Alahi

Cross-validation “command center”

A. Alahi

NN TIPS & TRICKS

A. Alahi

- Preprocess data — **Normalize each dimension of the input**
- Design an architecture — **Start from state-of-the-art designs**
  - Loss function — **Select task appropriate loss function (pytorch losses)**
  - Layers type — **"Conv" for signals such as images, and more recently Transformers**
  - #layers, #Filters — **Start with popular designs**
  - Activation function — **ReLu, elu**
- Sanity check — **Only once**
  - Gradient check — **Analytical VS num grad (while turning all regularization off) *(fix rand seed)***
  - Loss dynamics — **At chance or when regularization is increased**
  - Overfit subset of the data — **Use small subset of the data (while regularization is off)**
- Initialize weight "smartly" — **Pytorch/Tensorflow initialization**
- Batch normalization — **Try with and without. Test also Layer norm, group norm**
- Regularization strategies — **Data augment (Augmix, RandAugment, MixUp), no dropout in ResNet, DropBlock**
- Optimization strategy — **AdamW, Adam, SGD+Nesterov momentum, warm-up, learning rate scheduler**
- Monitor learning process — **Check loss, training/validation accuracy, ratio of weights updates (e.g., Wandb)**
- Hyperparameters search — **Random sample in log space (wide to coarse) (e.g., ray autotuner)**

NN TIPS & TRICKS

A. Alahi

- Preprocess data — **Normalize each dimension of the input**
- Design an architecture — **Start from state-of-the-art designs**
  - Loss function — **Select task appropriate loss function (pytorch losses)**
  - Layers type — **"Conv" for signals such as images, and more recently Transformers**
  - #layers, #Filters — **Start with popular designs**
  - Activation function — **ReLu, elu**
- Sanity check — **Only once**
  - Gradient check — **Analytical VS num grad (while turning all regularization off) *(fix rand seed)***
  - Loss dynamics — **At chance or when regularization is increased**
  - Overfit subset of the data — **Use small subset of the data (while regularization is off)**
- Initialize weight "smartly" — **Pytorch/Tensorflow initialization**
- Batch normalization — **Try with and without. Test also Layer norm, group norm**
- Regularization strategies — **Data augment (Augmix, RandAugment, MixUp), no dropout in ResNet, DropBlock**
- Optimization strategy — **AdamW, Adam, SGD+Nesterov momentum, warm-up, learning rate scheduler**
- Monitor learning process — **Check loss, training/validation accuracy, ratio of weights updates (e.g., Wandb)**
- Hyperparameters search — **Random sample in log space (wide to coarse) (e.g., ray autotuner)**
- Model ensembles

Model Ensembles

1. Train multiple independent models
2. At test time average their results

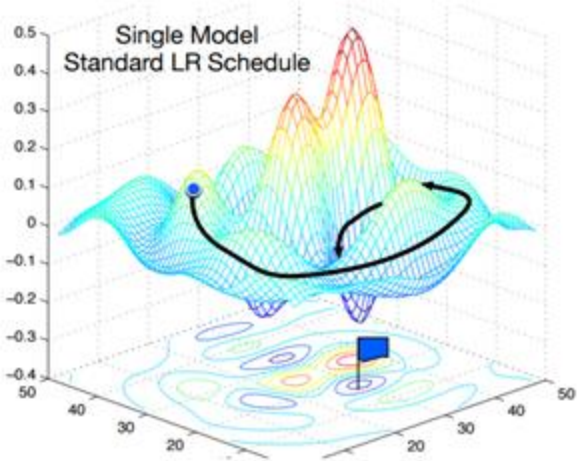
- Enjoy 2% extra performance

A. Alahi

Model Ensembles: Tips and Tricks

A. Alahi

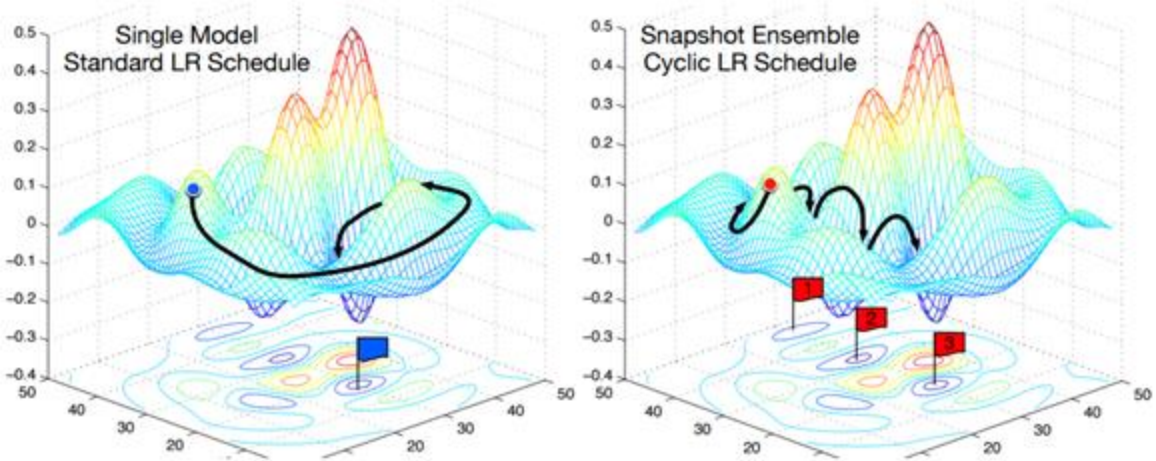
- Instead of training independent models, use multiple snapshots of a single model during training!



Loshchilov and Hutter, “SGDR: Stochastic gradient descent with restarts”, arXiv 2016
Huang et al, “Snapshot ensembles: train 1, get M for free”, ICLR 2017
Figures copyright Yixuan Li and Geoff Pleiss, 2017. Reproduced with permission.

Model Ensembles: Tips and Tricks

- Instead of training independent models, use multiple snapshots of a single model during training!



Loshchilov and Hutter, “SGDR: Stochastic gradient descent with restarts”, arXiv 2016
Huang et al, “Snapshot ensembles: train 1, get M for free”, ICLR 2017
Figures copyright Yixuan Li and Geoff Pleiss, 2017. Reproduced with permission.

NN TIPS & TRICKS

A. Alahi

- Preprocess data — **Normalize each dimension of the input**
- Design an architecture — **Start from state-of-the-art designs**
  - Loss function — **Select task appropriate loss function (pytorch losses)**
  - Layers type — **"Conv" for signals such as images, and more recently Transformers**
  - #layers, #Filters — **Start with popular designs**
  - Activation function — **ReLu, elu**
- Sanity check — **Only once**
  - Gradient check — **Analytical VS num grad (while turning all regularization off) *(fix rand seed)***
  - Loss dynamics — **At chance or when regularization is increased**
  - Overfit subset of the data — **Use small subset of the data (while regularization is off)**
- Initialize weight "smartly" — **Pytorch/Tensorflow initialization**
- Batch normalization — **Try with and without. Test also Layer norm, group norm**
- Regularization strategies — **Data augment (Augmix, RandAugment, MixUp), no dropout in ResNet, DropBlock**
- Optimization strategy — **AdamW, Adam, SGD+Nesterov momentum, warm-up, learning rate scheduler**
- Monitor learning process — **Check loss, training/validation accuracy, ratio of weights updates (e.g., Wandb)**
- Hyperparameters search — **Random sample in log space (wide to coarse) (e.g., ray autotuner)**
- Model ensembles — **Average top 10 performing models**

NN TIPS & TRICKS

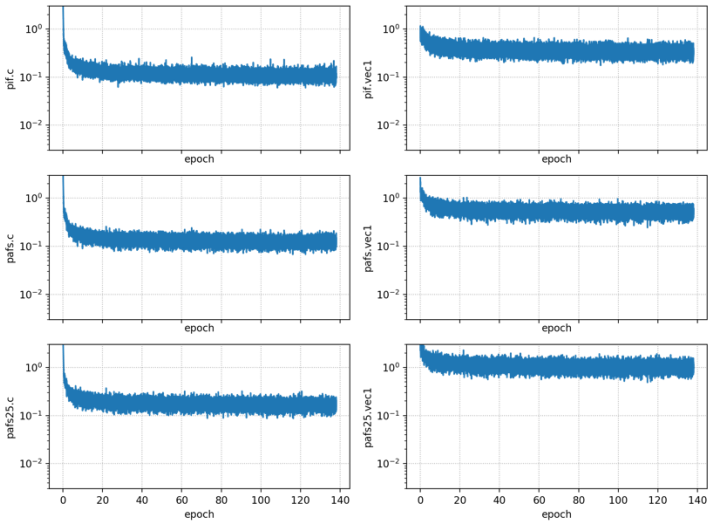
A. Alahi

- Preprocess data — **Normalize each dimension of the input**
- Design an architecture — **Start from state-of-the-art designs**
  - Loss function — **Select task appropriate loss function (pytorch losses)**
  - Layers type — **"Conv" for signals such as images, and more recently Transformers**
  - #layers, #Filters — **Start with popular designs**
  - Activation function — **ReLu, elu**
- Sanity check — **Only once**
  - Gradient check — **Analytical VS num grad (while turning all regularization off) *(fix rand seed)***
  - Loss dynamics — **At chance or when regularization is increased**
  - Overfit subset of the data — **Use small subset of the data (while regularization is off)**
- Initialize weight "smartly" — **Pytorch/Tensorflow initialization**
- Batch normalization — **Try with and without. Test also Layer norm, group norm**
- Regularization strategies — **Data augment (Augmix, RandAugment, MixUp), no dropout in ResNet, DropBlock**
- Optimization strategy — **AdamW, Adam, SGD+Nesterov momentum, warm-up, learning rate scheduler**
- Monitor learning process — **Check loss, training/validation accuracy, ratio of weights updates (e.g., Wandb)**
- Hyperparameters search — **Random sample in log space (wide to coarse) (e.g., ray autotuner)**
- Model ensembles — **Average top 10 performing models**
- Sub-loss magnitude ~ 1 — **Numerical stability**

Multi-Task Losses

A. Alahi

- Often necessary to do multiple tasks simultaneously. Below are training curves showing the value of their sub-losses for 6 sub-tasks.
- Each sub-task can be multiplied by a constant, the “weight” of the sub-task.
- Sometimes, there is no natural weight. Then, a good initial weight guess is to make every sub-loss ~1.



NN TIPS & TRICKS

A. Alahi

- Preprocess data — **Normalize each dimension of the input**
- Design an architecture — **Start from state-of-the-art designs**
  - Loss function — **Select task appropriate loss function (pytorch losses)**
  - Layers type — **"Conv" for signals such as images, and more recently Transformers**
  - #layers, #Filters — **Start with popular designs**
  - Activation function — **ReLu, elu**
- Sanity check — **Only once**
  - Gradient check — **Analytical VS num grad (while turning all regularization off) *(fix rand seed)***
  - Loss dynamics — **At chance or when regularization is increased**
  - Overfit subset of the data — **Use small subset of the data (while regularization is off)**
- Initialize weight "smartly" — **Pytorch/Tensorflow initialization**
- Batch normalization — **Try with and without. Test also Layer norm, group norm**
- Regularization strategies — **Data augment (Augmix, RandAugment, MixUp), no dropout in ResNet, DropBlock**
- Optimization strategy — **AdamW, Adam, SGD+Nesterov momentum, warm-up, learning rate scheduler**
- Monitor learning process — **Check loss, training/validation accuracy, ratio of weights updates (e.g., Wandb)**
- Hyperparameters search — **Random sample in log space (wide to coarse) (e.g., ray autotuner)**
- Model ensembles — **Average top 10 performing models**
- Sub-loss magnitude ~ 1 — **Numerical stability**
- Pretrained networks — **Especially helpful when own training data is small**

NN TIPS & TRICKS

A. Alahi

- “You need a lot of a data if you want to train/use Deep NNs”

IMAGENET

www.image-net.org

22K categories and **14M** images

- Animals
  - Bird
  - Fish
  - Mammal
  - Invertebrate
- Plants
  - Tree
  - Flower
- Food
- Materials
- Structures
- Artifact
  - Tools
  - Appliances
  - Structures
- Person
- Scenes
  - Indoor
  - Geological Formations
- Sport Activities

Deng, Dong, Socher, Li, Li, & Fei-Fei, 2009

IMAGENET Large Scale Visual Recognition Challenge

Steel drum

The Image Classification Challenge:

1,000 object classes

1,431,167 images

Output:
Scale
T-shirt
<u>Steel drum</u>
Drumstick
Mud turtle

Output:
Scale
T-shirt
Giant panda
Drumstick
Mud turtle

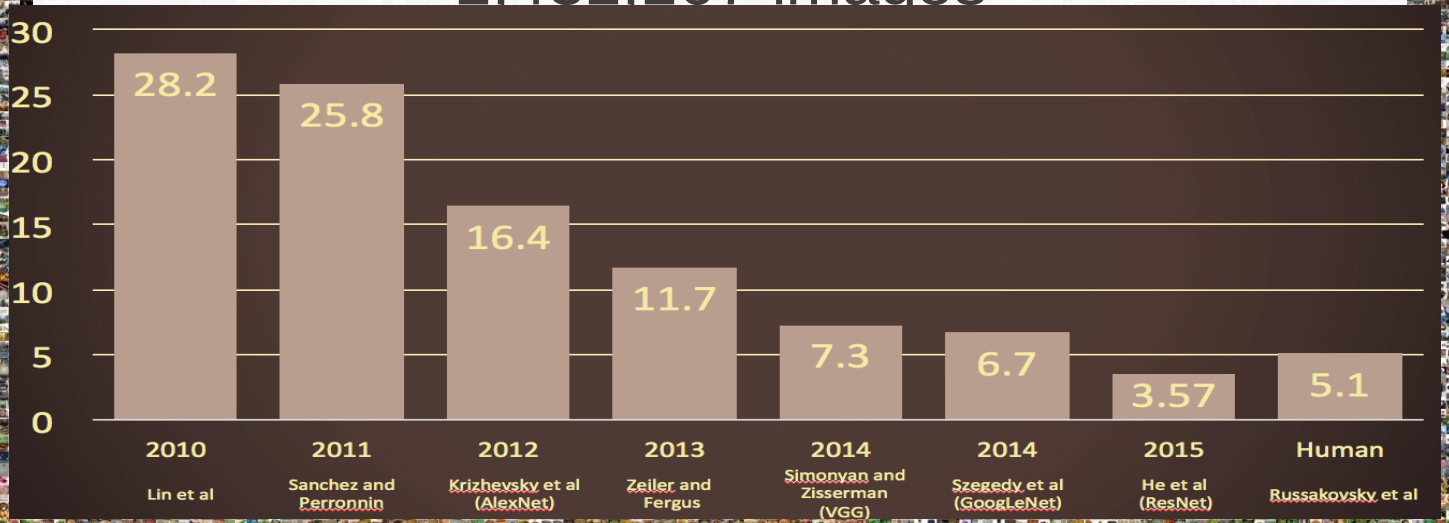
Russakovsky et al., 2014

IMAGENET Large Scale Visual Recognition Challenge

The Image Classification Challenge:

1,000 object classes

1,431,167 images



Russakovsky et al., 2014

NN TIPS & TRICKS

- “You need a lot of a data if you want to train/use Deep NNs”

- What if you don’t?
- Transfer Learning

Transfer Learning with CNNs

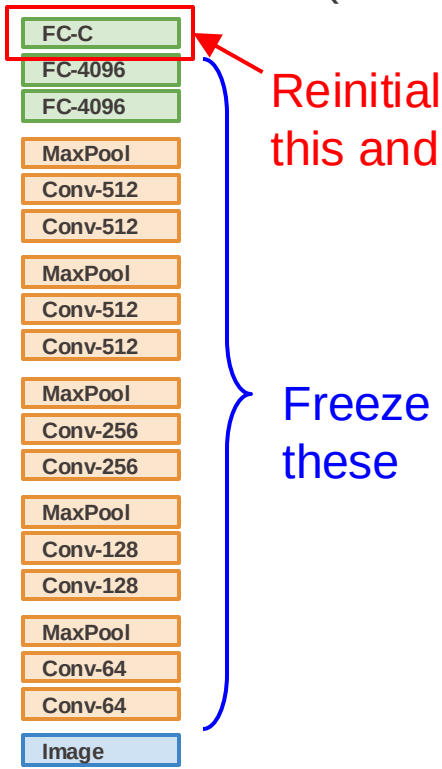
A. Alahi

1. Train on Imagenet

Transfer Learning with CNNs

1. Train on Imagenet

2. Small Dataset (C classes)



Reinitialize this and train

Freeze these

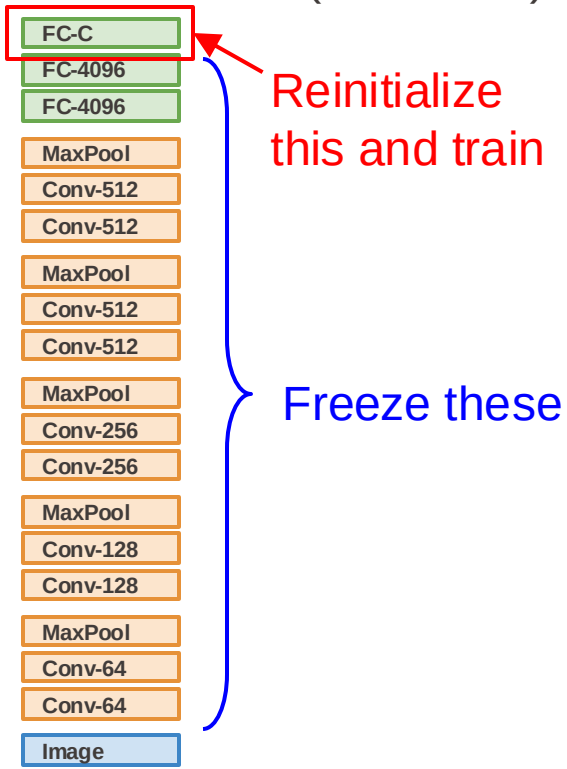
A. Alahi

Transfer Learning with CNNs

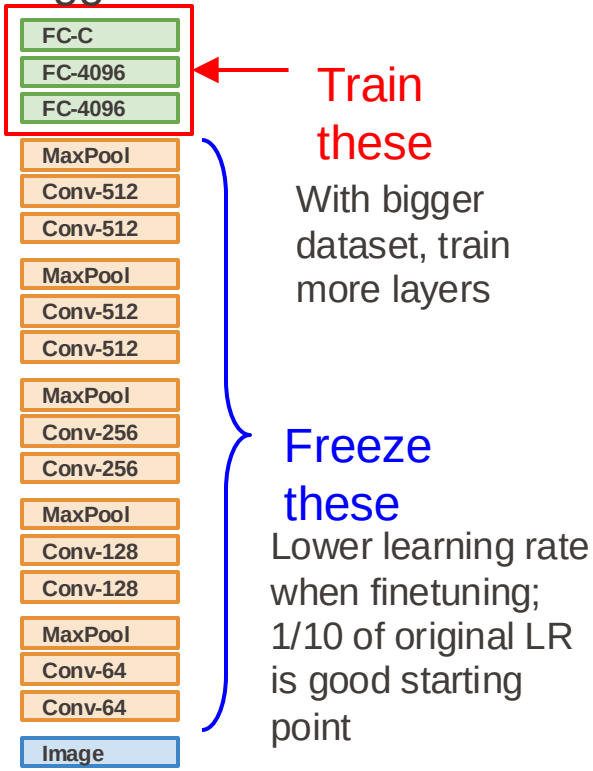
A. Alahi

1. Train on Imagenet

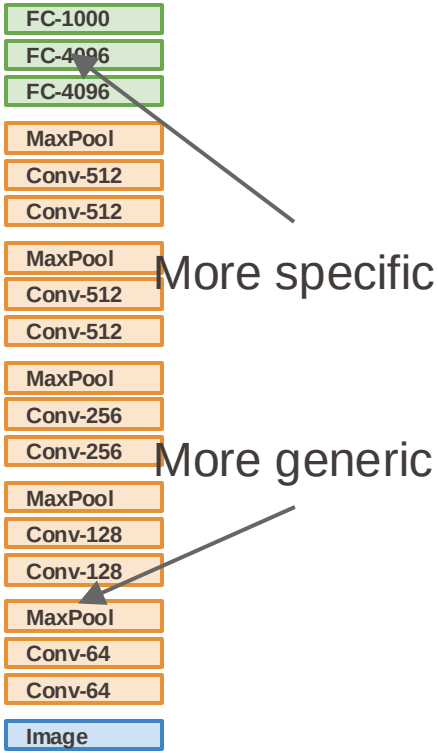
2. Small Dataset (C classes)



3. Bigger dataset

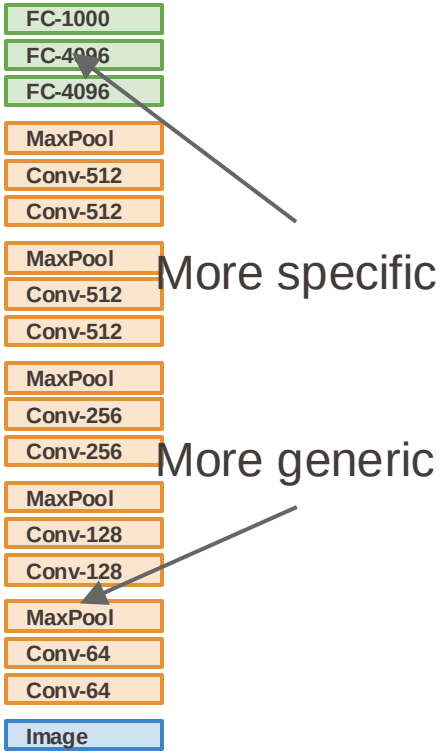


A. Alahi



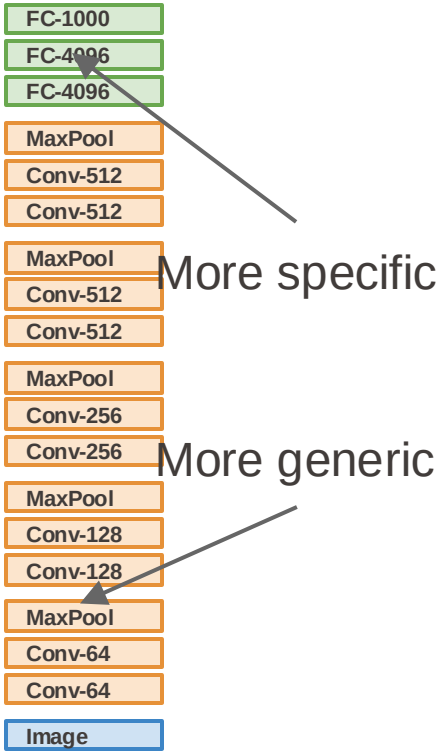
	very similar dataset	very different dataset
very little data	?	?
quite a lot of data	?	?

A. Alahi



	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer	?
quite a lot of data	Finetune a few layers	?

A. Alahi



	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer	You're in trouble… Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

Takeaway

- When you have some a small dataset

1. Find a very large dataset that has similar data, train a big network there
2. Transfer learn to your dataset

- Deep learning frameworks provide a “Model Zoo” of pretrained models so you don’t need to train your own
- PyTorch: https://github.com/pytorch/vision

- https://teachablemachine.withgoogle.com/train/image

NN TIPS & TRICKS

A. Alahi

- Preprocess data › **Normalize each dimension of the input**
- Design an architecture › **Start from state-of-the-art designs**
  - Loss function › **Select task appropriate loss function (pytorch losses)**
  - Layers type › **"Conv" for signals such as images, and more recently Transformers**
  - #layers, #Filters › **Start with popular designs**
  - Activation function › **ReLu, elu**
- Sanity check › **Only once**
  - Gradient check › **Analytical VS num grad (while turning all regularization off) *(fix rand seed)***
  - Loss dynamics › **At chance or when regularization is increased**
  - Overfit subset of the data › **Use small subset of the data (while regularization is off)**
- Initialize weight "smartly" › **Pytorch/Tensorflow initialization**
- Batch normalization › **Try with and without. Test also Layer norm, group norm**
- Regularization strategies › **Data augment (Augmix, RandAugment, MixUp), no dropout in ResNet, DropBlock**
- Optimization strategy › **AdamW, Adam, SGD+Nesterov momentum, warm-up, learning rate scheduler**
- Monitor learning process › **Check loss, training/validation accuracy, ratio of weights updates (e.g., Wandb)**
- Hyperparameters search › **Random sample in log space (wide to coarse) (e.g., ray autotuner)**
- Model ensembles › **Average top 10 performing models**
- Sub-loss magnitude ~ 1 › **Numerical stability**
- Pretrained networks › **Especially helpful when own training data is small**

- + Data distribution
- + Gradient flow
- **+ Visualize your input data, intermediate values, results**

References

- Book: Deep Learning by Ian Goodfellow, Yoshua Bengio, Aaron Courville
  (http://www.deeplearningbook.org/)

- Class on CNN: http://cs231n.stanford.edu/

- Good tuto on gradient check:

http://cs231n.github.io/neural-networks-3/

- On gradient based optimization methods:

http://ruder.io/optimizing-gradient-descent/

Additional materials

- CNN demo on CIFAR-10:

http://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html